



TABLETOP ROLEPLAYING SYSTEM

RGB System

Core Rules

V2.0 - ENGLISH
2026-08-28

MIT
4AED7BB

Colophon

TITLE

RGB System - Core Rules

VERSION

v2.0 (2026-08-28)

LANGUAGE

English (en)

AUTHORS

RGB System contributors

LICENSE

MIT

SOURCE

<https://github.com/SergioLacerda/rpg-system-rgb> @ 4aed7bb

GENERATOR

MkDocs + WeasyPrint

RGB System	6
Quick Start	9
System Overview	13
One Page Rules	15
Overview	19
Attributes	20
Character Creation	23
Progression	26
Skills and Abilities	29
Overview	34
Attack and Defense	35
Combat Decision Model	38
Damage Model	42
Movement	46
Overview	50
Armor	51
Gear	54
Shields	57
Weapons	60
Overview	61

Character Sheet	63
Combat Example	66
Combat Walkthrough	70
Extra	74
Gameplay Example	79
Gameplay Loop	83
Damage Interaction Model	86
Architecture Notes	92
System Engine	100
Glossary	106
ADR-001: AI-First Documentation Authority	108
ADR-002: RGB Core V2 Design Control	110
ADR-003: Documentation L10n Maintenance Pattern	111
ADR-004: Modular Monolith In A Single Monorepo	113
ADR-005: Zero External Dependency Posture	115
ADR-006: cmd/rgb As The Unified CLI	118
ADR-007: Bundle Expansion Requires A Consumer Contract First	121
ADR-008: Retire The Go HTML/Print/PDF Publication Pipeline	124
ADR-009: Retain The MkDocs/Python Publication Exception Until Replacement Parity	126
ADR-010: Go-Owned Publication Without Python	129
ADR-011: Split License Model	131
ADR-012: MIT-Only License	133

ADR-013: Skill Distribution Via The Existing Publication Pipeline	135
ADR-014: RGB Specialist Bundled References Snapshot	138
ADR-015: Skill Runtime Split — LLM Instruction Layer vs. Go Component Layer	140
ADR-016: Reinstate A Narrow External PDF Authoring Exception	143
Architecture Decision Records	146
Explosives	147
Firearms	150
Melee Weapons	153
Penetration	155
Weapon Ranges	157

RGB System — Documentation Index

This index organizes the complete **RGB System** documentation in English.

Its purpose is to provide a central navigation hub for all system modules.

RGB System

The **RGB System** is a tabletop RPG designed around simple rules and tactical combat.

The system uses three core vectors:

Vector	Name	Function
-----R	Red	attacks, power, strength checks, accuracy, damage
G	Green	agility, mobility, evasion, reaction speed
B	Blue	energy shield, special protection, intellect, energy

The system was designed around four principles:

- structural simplicity
- tactical depth
- modularity
- scalability

It supports multiple genres:

- modern campaigns
- fantasy
- science fiction
- superpower settings

Introduction

Documents presenting the fundamental concepts of the system.

- [System Overview](#)

- [Quick Start](#)
- [One Page Rules](#)

Core System

Defines the foundation of the RGB system.

- [Attributes](#)
- [Character Creation](#)
- [Skills and Abilities](#)

Combat

Rules governing combat encounters.

- [Movement](#)
- [Attack and Defense](#)
- [Damage Model](#)
- [Combat Decision Model](#)

Equipment

Equipment affecting survivability and tactical choices.

- [Armor](#)
- [Shields](#)
- [General Gear](#)

Weapons

Weapon types and related mechanics.

- [Firearms](#)
- [Melee Weapons](#)
- [Explosives](#)
- [Penetration](#)
- [Weapon Ranges](#)

Reference

Supporting material and gameplay examples.

- [Character Sheet](#)

- [Combat Example](#)
- [Gameplay Example](#)
- [Gameplay Loop](#)
- [RGB System Engine](#)

Architecture Decisions

Project-wide ADRs are maintained separately from the public rules Library.

Documentation Structure

```
en/
├─ introduction/
├─ core/
├─ combat/
├─ equipment/
├─ weapons/
└─ reference/
```

Each folder contains its own **README.md** acting as a local index.

RGB System – Quick Start

This quick start guide explains the core ideas of the RGB System and how to begin playing in just a few minutes.

1. The RGB Vectors

Characters in the RGB System are defined by three vectors:

```
| Vector | Name | Function |  
|---|  
| R | Red | pressure, impact, disruption |  
| G | Green | relation, movement, reaction |  
| B | Blue | preservation, shields, stabilization |
```

These three values define how a character performs in combat and interaction.

2. Creating a Character

A starting character typically receives:

7 points

Players distribute these points between **R, G and B**.

Example:

```
R = 3  
G = 2  
B = 2
```

3. Character Durability

Durability is calculated using simple formulas.

```
Health = 4 + R + B  
Shield = B × 3
```

Example:

```
R = 3  
B = 2  
Health = 9  
Shield = 6
```

4. Basic Turn Structure

During combat a character typically gets:

```
Movement (G × 2 meters, free)  
+  
1 Action  
+  
1 Minor Action (optional)
```

Examples:

Actions:

- attack
- defend
- use ability

Minor Actions:

- reload weapon
- change position slightly
- interact with environment

5. Combat Decisions

Combat revolves around three tactical choices:

```
Attack  
Move  
Defend
```

These map directly to the RGB vectors:

```
Press → R  
Reposition → G  
Sustain → B
```

Players naturally emphasize different strategies depending on their vector distribution.

6. Damage Resolution

When an attack hits, damage follows this sequence:

```
Hit or Contact Check  
↓  
Impact Source  
↓  
Penetration  
↓  
Armor Reduction  
↓  
Shield Absorption  
↓  
Remaining Damage → Character
```

Armor reduces damage **per attack**, while shields absorb **remaining accumulated damage**.

7. Example Combat Situation

A character with:

```
R = 3  
G = 2  
B = 2
```

faces an enemy.

The player must choose between:

- increasing damage output
- repositioning
- reinforcing defense

Each decision corresponds to one of the RGB vectors.

8. Gameplay Loop

A typical RGB session follows this structure:

Character Creation

↓

Exploration

↓

Combat Encounter

↓

Recovery

↓

Story Progression

This loop repeats as the story develops.

Learn More

For deeper understanding see:

- Core Rules → [Core Rules](#)
- Combat System → [Combat System](#)
- Equipment → [Equipment](#)
- Weapons → [Weapons](#)
- Gameplay Loop → [Gameplay Loop](#)

System Overview

The RGB System is built around three fundamental vectors:

- **R (Red)** — power, attacks and damage
- **G (Green)** — agility, mobility and reaction
- **B (Blue)** — shields, energy and special resistance

These vectors define how characters interact with the world and with combat.

System Structure

The system is divided into several major sections.

Core Rules

The core rules define the RGB vectors and character creation.

- [Attributes](#)
- [Character Creation](#)
- [Skills and Abilities](#)

Combat System

Combat rules define how characters interact during encounters.

- [Attack and Defense](#)
- [Combat Decision Model](#) → [Damage Model](#) → [Movement](#)

Equipment

Equipment provides defensive and tactical options.

- [Armor](#)
- [Shields](#)
- [Gear](#)

Weapons

Weapons determine how damage is applied during combat.

- [Firearms](#)
- [Melee Weapons](#)
- [Explosives](#)

Reference

Examples and supporting material for gameplay.

- [Character Sheet](#)
- [Combat Example](#)
- [Gameplay Example](#)

Gameplay Flow

A typical gameplay interaction follows this structure:

Character Creation

- Equipment selection
- Tactical positioning
- Combat resolution
- Recovery and progression

Additional Design Notes

Further technical explanations of the RGB system can be found here:

- [RGB System Architecture](#)

RGB Interaction Model

The system can be visualized as a triangle:

```
      G
    mobility / reaction

R ----- B
power / damage   shield / energy
```

Each vector represents a different strategy for interacting with combat.

RGB System — One Page Rules

A minimal reference to start playing the **RGB RPG System** immediately.

Core Concept

Characters are defined by three vectors:

Vector	Name	Function
R	Red	pressure, impact, disruption
G	Green	relation, mobility, reaction
B	Blue	preservation, shields, stabilization

Players distribute **7 starting points** between R, G and B.

Example:

R = 3
G = 2
B = 2

Character Durability

Health = 4 + R + B

Shield = B × 3

Health represents physical endurance plus preservation.
Shield represents energy or special protection.

Turn Structure

Each turn a character gets:

Movement (G × 2 meters, free)
+
1 Action
+
1 Minor Action

Examples:

Actions:

- attack
- defend
- use ability

Minor Actions:

- reload
- small reposition
- interact with object

Tactical Choices

Most combat decisions fall into three categories:

Press → R
Reposition → G
Sustain → B

This creates three combat strategies:

Strategy	Vector
Striker	R
Skirmisher	G
Guardian	B

Hybrid builds are possible.

Attack Resolution

An attack succeeds if the attacker overcomes the defender according to the combat rules.

If the attack succeeds, damage is resolved.

Damage Flow

```
Hit or Contact Check
↓
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

Definitions:

Armor → reduces damage per hit

Shield → absorbs accumulated damage

Example Damage

Weapon Damage: 7

Penetration: 2

Armor: 4

```
Effective Armor = Armor - Penetration
Effective Armor = 2
Final Damage = 7 - 2 = 5
```

If a shield exists, damage is absorbed by the shield first.

Gameplay Loop

A typical session follows:

```
Character Creation
↓
Exploration
↓
Combat
↓
Recovery
↓
Story Progression
```

RGB Tactical Model

G
mobility / reaction

R - B
power / damage shield / defense

R → deal damage or change the source of pressure

G → avoid damage or change relation to pressure

B → absorb damage or preserve continuity under pressure

Design Philosophy

The RGB system focuses on:

- simple numeric relationships
- tactical combat decisions
- modular rules
- adaptable settings

The same rules can support:

- modern campaigns
- fantasy worlds
- science fiction
- superpowered settings

Core Rules

This section defines the fundamental mechanics of the RGB System.

The core rules describe how characters are built and how the RGB vectors interact with the game world.

Topics

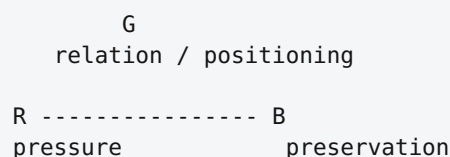
- [Attributes](#)
Explains the RGB vectors and how they define character capabilities.
- [Character Creation](#)
Step-by-step guide to building a character using the RGB system.
- [Progression](#)
Character advancement rules: vector growth and alternative advancement choices.
- [Skills and Abilities](#)
Optional abilities and meta-skills that expand character capabilities.

← [Back to README](#)

Attributes

The RGB System is built around three fundamental vectors that define how characters interact with the world and with combat.

These vectors form the **RGB Tactical Triangle**, where each attribute represents a different strategic approach.



- **R (Red)** changes the source of pressure through force, impact, disruption, or transformation.
- **G (Green)** changes a character's relation to pressure through movement, timing, evasion, reach, or positioning.
- **B (Blue)** preserves continuity under pressure through endurance, blocking, shields, stabilization, resistance, or protection.

Together, these vectors create the core tactical dynamics of the RGB System.

RGB Vectors

Red (R)

Represents **pressure and physical impact**.

Typical uses:

- physical strength;
- attack pressure;
- impact force;
- interruption;
- breaking, forcing, or transforming the source of danger.

Characters with high **R** tend to favor aggressive combat strategies.

Green (G)

Represents **relation, timing, and positioning**.

Typical uses:

- mobility;
- reflexes;
- evasion;
- tactical repositioning;
- range, timing, hiding, detection, and contact control.

Characters with high **G** excel at controlling distance and avoiding attacks.

Blue (B)

Represents **preservation under pressure**.

Typical uses:

- endurance;
- energy control;
- blocking and stabilization;
- magical or technological shields;
- resistance to damage;
- protection of allies or positions.

Characters with high **B** specialize in survival and defensive tactics.

Tactical Interpretation

The three vectors correspond to the main combat decisions in the RGB system:

```
Press      → R
Reposition → G
Sustain    → B
```

This relationship forms the foundation of the RGB combat philosophy. Players naturally emphasize different strategies depending on their vector distribution.

Examples:

```
| Character Style | Vector Focus |
|-----|-----|
Heavy attacker | High R |
Mobile skirmisher | High G |
Defensive guardian | High B |
```

Hybrid builds are also possible by combining vectors.

Power Scale

The following scale provides a general interpretation of vector values.

Value	Interpretation
1	weak
3	trained human
5	professional
7	elite
9	exceptional
10	extraordinary

Vectors may exceed **10** through abilities, equipment, or special powers depending on the campaign setting.

Design Philosophy

The RGB attribute model keeps character creation simple while supporting tactical depth.

Each vector represents a different combat philosophy:

R → deal damage or change the pressure source
G → avoid damage or change relation to pressure
B → absorb damage or preserve continuity under pressure

Because every character must choose how to distribute their points between these vectors, the system naturally produces diverse character builds.

See also:

- [Character Creation](#)

← [Back to README](#)

Character Creation

Character creation in the **RGB System** is intentionally simple.

Players distribute points between the three RGB vectors to define how their character performs in combat and interaction.

G
relation / positioning

R ----- B
pressure preservation

- **R (Red)** → pressure, force, attack, disruption, and impact
- **G (Green)** → movement, timing, evasion, range, and positioning
- **B (Blue)** → endurance, blocking, stabilization, shields, and protection

This distribution defines the character's tactical style.

Starting Points

Characters begin with:

7 points

These points must be distributed between:

R (Red)
G (Green)
B (Blue)

Example:

R = 3
G = 2
B = 2

Different distributions naturally create different combat roles.

```
| Style | Vector Focus |  
|-----|-----|  
Heavy attacker | High R |  
Mobile skirmisher | High G |  
Defensive guardian | High B |
```

Hybrid builds are also possible.

Character Progression

As characters gain experience they become stronger. See [Progression](#) for the full advancement rules, including the per-advancement choice between vector growth and alternatives such as new abilities, specialization, or new resources.

Health

Character durability uses both pressure tolerance and preservation.

```
Health = 4 + R + B
```

Example:

```
R = 3  
B = 2  
Health = 9
```

This prevents **R** from owning both offense and durability by itself. Higher **R** still helps characters withstand physical pressure, while higher **B** represents preservation, stability, and defensive endurance.

Quick Reference

```
Attack      : margin = attacker R - defender G  
Movement   : G × 2 meters  
Health      : 4 + R + B  
Shield      : B × 3
```

These formulas summarize the core mechanical interactions of the RGB system.

Design Philosophy

Character creation in RGB follows three principles:

- **simplicity** — few numbers define the character

- **tactical diversity** — different vector distributions create different play styles
- **modularity** — abilities and equipment can expand the system without changing its core rules

Because every character distributes points between the same three vectors, the system naturally produces balanced and varied builds.

See also:

- [Attributes](#)
- [Progression](#)
- [Combat](#)
- [Movement](#)

← [Back to README](#)

Progression

As characters gain experience, they grow stronger. Progression in the **RGB System** is built on a small, fixed advancement budget per level, which a player may spend either as raw vector growth or, when the campaign uses the Skills and Abilities module, as one of several alternative advancement choices.

This document is the canonical source for character progression. It replaces the short summary previously kept inline in [Character Creation](#), which now links here.

Advancement Budget

+2 vector points per level

The Game Master determines when characters gain a level, depending on the campaign. This baseline never changes — every advancement choice below spends the same fixed budget, it does not add to it.

Advancement Choices

By default, the full budget becomes vector points. When the campaign allows the [Skills and Abilities](#) module, the player may instead spend a level's advancement on one alternative below. Exactly one choice applies per level; choices do not stack within the same level.

Choice	Effect
Vector growth (default)	+2 points, distributed across R, G, and B as the player chooses
New ability	Gain one ability whose tier and requirements (see the Ability Contract in Skills and Abilities) the character currently meets
Ability improvement	Reduce one known ability's cost , extend its duration , or raise its limits ceiling by one step, at the Game Master's discretion
Specialization	Commit to one vector's ability tree (Red, Green, or Blue): future new-ability choices in that tree ignore one tier of requirements
New resource	Gain a campaign-defined resource pool (e.g. an extra reaction charge, a callback-recovery buffer) sized by the Game Master
New reaction	Gain access to one reaction procedure not otherwise available at the character's current tier
New state or maneuver access	Gain the ability to declare one tactical state or maneuver (see Attack and Defense , Movement) that was previously restricted or unavailable

Specialization In Practice

Specialization is the one choice with a lasting structural effect: it commits a character to a vector's identity rather than granting an immediate benefit. A Red-specialized character does not gain a Red ability from specializing alone — specialization changes how *future* Red abilities are gated, per the Ability Contract's **requirements** field. This keeps specialization meaningful without requiring new resource tracking beyond what the Ability Contract already defines.

Design Rationale

The RGB System V2 initial analysis (**base_project** intake, §7.5) left this choice open: pure numeric vector growth, or a per-advancement choice among several options. This document resolves it as a **hybrid**, not a new invention:

- The pure-numeric baseline (**+2 vector points per level**) was already canonical, stated in Character Creation before this document existed. This document keeps it unchanged as the default — no existing example or rule that assumed pure numeric growth is invalidated.
- [Skills and Abilities](#) already showed abilities gated by level in its worked examples (**Level 3 → +1 R to ranged attacks**, **Level 6 → Double power strike**, **Level 9 → Area impact attack**) — a per-advancement ability choice was already implied in practice, just never formalized as a documented option alongside vector growth. This document only makes that existing pattern explicit and consistent with the Ability Contract's

tier / **requirements** fields, which already exist specifically to gate ability access by advancement.

- A hybrid keeps the system's stated design philosophy intact: character creation and progression both favor **simplicity** (one fixed budget, one choice per level) while still supporting **tactical diversity** (the campaign, via the optional Skills and Abilities module, can offer more than pure numbers without adding new subsystems).

Example

A level-6 character with **R=5, G=3, B=4** and the Skills and Abilities module active can choose:

Vector growth → R=7, G=3, B=4 (or any +2 split)
New ability → gain one ability meeting tier ≤ character's current tier
Specialization → commit to Red, Green, or Blue for future ability gating

The player picks one; the choice does not carry over or accumulate.

See also:

- [Character Creation](#)
- [Skills and Abilities](#)
- [Attributes](#)

← [Back to README](#)

Skills and Abilities

Skills represent special capabilities beyond the normal actions of the RGB system. They allow characters to expand their abilities depending on the campaign setting.

Skills are **optional modules** and may represent:

- martial techniques
- supernatural powers
- advanced technology
- magic systems

The Game Master determines whether these abilities are available in the campaign.

RGB Ability Structure

Abilities in the RGB system are organized around the **three core vectors**.

```
      G
    mobility / positioning

R ----- B
power / damage   shield / defense
```

Each vector naturally forms a **branch of abilities**.

- **R (Red)** → pressure, impact, interruption, and transformation abilities
- **G (Green)** → movement, timing, evasion, range, and positioning abilities
- **B (Blue)** → preservation, stabilization, block, shield, and resistance abilities

This structure allows the system to scale naturally with different settings.

Red Ability Tree (R)

Red abilities enhance **offensive power and physical force**.

Typical abilities:

- Double Power

- Delayed Strike
- Air Pressure Attack
- Power Strike
- Armor Break

These abilities increase a character's ability to deal damage.

Example progression:

Level 3 → +1 R to ranged attacks (Precision)
Level 6 → Double power strike
Level 9 → Area impact attack

Green Ability Tree (G)

Green abilities focus on **mobility, speed, and tactical positioning**.

Typical abilities:

- Enhanced Reflexes
- Momentary Acceleration
- Phantom Step (short teleport-like movement)
- Automatic Dodge
- Supernatural Sprint
- Enhanced Jump
- Silent Movement
- Movement Distortion
- Supernatural Evasion

These abilities allow characters to control space and avoid damage.

Blue Ability Tree (B)

Blue abilities represent **energy manipulation, endurance, and defensive systems**.

Typical abilities:

- Energy Shield
- Stunning Shield
- Intelligent Shield / Living Armor
- Invisibility
- Clones
- Energy Absorption

These abilities improve survivability, continuity, and energy-based defenses.

Ability Contract

Every ability must declare the following fields before it can be treated as canonical, validated, bundled, or used by a Specialist:

Field	Purpose
<code>id</code>	stable identifier
<code>name</code>	table-facing name
<code>vector</code>	primary RGB vector
<code>tier</code>	level, rank, or access tier
<code>requirements</code>	prerequisites
<code>action_type</code>	action, reaction, passive, stance, or special timing
<code>cost</code>	vector points, shield, callback, item charge, or other cost
<code>range</code>	target or area reach
<code>duration</code>	instant, round, scene, persistent, or conditional
<code>effect</code>	mechanical effect
<code>limits</code>	per turn, per combat, cooldown, or narrative restriction
<code>tags</code>	classification tags
<code>source_status</code>	proposed, optional, canonical, deprecated, or example

Abilities that lack these fields are examples or design notes, not canonical rules.

Martial Factor (Optional Module)

The **Martial Factor** introduces advanced defensive techniques.

Examples:

- Complete Defense
- Special Absorption
- Defensive stance techniques

The Game Master decides whether these techniques require:

- an action
- a reaction

- a defensive stance

Blue Factor (Optional Module)

In campaigns that allow **magic or meta abilities**, the Blue vector can manipulate energy-based defenses.

These abilities usually interact with the shield system:

$$\text{Shield} = B \times 3$$

Examples:

- magical shields
- energy barriers
- energy absorption
- advanced defensive fields

Callback System (Optional)

If the campaign includes powerful abilities, the Game Master may apply a **callback system** to represent the cost or fatigue caused by these abilities.

If callback exceeds **100%**, the character faints.

Intensity	Recovery
Light	hours
Moderate	days
Severe	weeks
Extreme	months

The Game Master may also apply permanent consequences depending on the situation.

Special Absorption

Special Absorption allows characters to reduce incoming damage using a declared resource or vector expression.

1 vector point spent → reduces 1 damage

Possible uses:

- **Red (R)** → physical impact absorption
- **Blue (B)** → energy or special damage absorption

The Game Master may adjust this ratio depending on the campaign.

Complete Defense

Complete Defense is a legacy name for a block or guard stance where the character prepares to receive pressure rather than evade it.

This stance also negates grapple attempts.

Complete Defense = Block procedure + declared absorption effect

Example Ability Categories

Mobility

- flight
- levitation
- teleportation

Perception

- night vision
- advanced sensors
- enhanced awareness

Technology

- remote device control
- hacking abilities
- drone interface

Creating RGB Abilities

Every ability should use the contract above. At minimum, it must define vector, cost, action type, effect, duration, limits, and source status.

This keeps abilities consistent with the RGB system.

See also:

- [Character Creation](#)
- [Attributes](#)

← [Back to README](#)

Combat System

The combat system defines how characters interact during encounters.

Topics

Movement

Rules for displacement, positioning, and terrain interaction.

→ [Movement](#)

Attack and Defense

Defines how attacks are resolved between characters.

→ [Attack and Defense](#)

Damage Model

Explains how damage flows through the system's layers.

→ [Damage Model](#)

Combat Decision Model

Shows how the RGB vectors drive tactical decisions.

→ [Combat Decision Model](#)

← [Back to README](#)

Attack And Defense

Initiative

More Green (G) points → acts first.

A surprise attack ignores initiative.

Resolution Philosophy

Core V2 resolves attacks with a **margin**. This keeps deterministic play possible while allowing richer outcomes than a flat hit or miss.

$$\text{Attack Margin} = \text{Attacker (R + modifiers)} - \text{Defender (G + modifiers)}$$

Use the margin as follows:

Margin	Result
3 or more	strong hit
1 to 2	hit
0	hit with cost or exposed follow-up
-1 to -2	miss with opportunity
-3 or less	clear miss

The older teaching shorthand remains valid:

$$\text{Attacker R} \geq \text{Defender G}$$

That shorthand means the attacker gets at least a basic hit when the margin is zero or higher.

Attack

Offensive pressure. The attacker uses Red (R) to change the target or the source of danger.

margin = Attacker (R + modifiers) - Target (G + modifiers)

If the result is a hit, continue to the damage model or to the declared non-damage consequence.

Defensive Procedures

Defense is no longer one combined number. Choose the procedure that describes what the defender is doing.

Procedure	Primary Owner	Purpose
Evade	G	avoid or alter contact
Reposition	G	change range, cover, or engagement
Block	B or equipment	intentionally receive pressure
Armor reduction	equipment	reduce impact per hit
Shield absorption	B-derived resource or equipment	absorb remaining damage
Protect ally	B, equipment, or ability	redirect or contain pressure
Interrupt	R	stop an action by applying pressure first
Counter	R or G by procedure	answer pressure through force or timing

Do not collapse these procedures into a generic

Defense = Armor + Shield

 formula. Armor and shield act later in the damage pipeline.

Evade

Evade uses G against the attacker's R.

Evade changes contact, timing, range, or position. If evasion succeeds, no damage is applied unless an ability or area effect says otherwise.

Block

Block uses B, equipment, or an ability to receive pressure intentionally. A block may reduce the attack margin, protect another target, or send the attack into armor and shield layers.

Grapple

Allows a character to immobilize the opponent or force movement during a turn.

A grapple action succeeds when:

Attacker (G) $\geq$ Target (G)

Attacker (R) $\geq$ Target (R)

The attacker may use meta-abilities.

The target may use pre-declared bonuses.

Damage

If an attack deals damage, use the damage model.

Damage Resolution

1. check hit
2. establish impact source
3. apply penetration
4. apply armor reduction
5. apply shield absorption
6. apply remaining health or state consequence

Meta Abilities or Magic

Magical Attack during combat

Each turn points are recovered to use spells or meta-abilities.

Regeneration = 1 B point per level

Magical Defense During Combat

Magical defense is a typed shield, block, resistance, or absorption effect. It must state which defensive procedure it modifies and whether it acts before or after armor.

See also:

- [Firearms](#)
- [Armor](#)

← [Back to README](#)

Combat Decision Model

The RGB System encourages players to make tactical decisions during combat.

Each RGB vector represents a different strategic approach.

RGB Tactical Triangle

Combat decisions can be visualized using the RGB triangle.

```
      G
    mobility / positioning

R ----- B
power / damage   shield / defense
```

Each vector represents a different combat strategy.

Combat Strategies

R – Offensive Strategy

Focus on direct damage and aggressive combat.

Typical actions:

- powerful attacks
- heavy weapons
- breaking enemy defenses

Advantages:

- high damage output
- faster enemy elimination

Weakness:

- lower survivability

G – Mobility Strategy

Focus on movement and positioning.

Typical actions:

- flanking enemies
- avoiding attacks
- controlling distance

Advantages:

- harder to hit
- tactical positioning

Weakness:

- lower direct damage

B – Defensive Strategy

Focus on survivability and protection.

Typical actions:

- shield usage
- defensive abilities
- absorbing enemy attacks

Advantages:

- high durability
- strong defensive capacity

Weakness:

- slower combat resolution

Tactical Interactions

The RGB system allows multiple combat styles.

Strategy	Vector Focus
-----	-----
Aggressive fighter	High R
Mobile skirmisher	High G
Defensive guardian	High B

Hybrid builds are also possible.

Examples:

```
| Build | Style |  
|-----|-----|  
R + G | fast striker |  
R + B | heavy warrior |  
G + B | evasive defender |
```

Combat Decision Flow

During combat a player typically chooses between three tactical options:

```
Press  
Reposition  
Sustain
```

These actions map naturally to the RGB vectors.

```
Press → R  
Reposition → G  
Sustain → B
```

Interaction with Damage Model

Combat decisions interact directly with the damage system.

```
Attack  
↓  
Impact Source  
↓  
Penetration  
↓  
Armor Reduction  
↓  
Shield Absorption  
↓  
Character
```

Players must choose whether to:

- change the source of pressure
- change relation to pressure
- preserve continuity under pressure

Design Philosophy

The RGB System creates a simple tactical model:

R → Deal damage or change the pressure source
G → Avoid damage or change relation to pressure
B → Absorb damage or preserve continuity

These three strategies form the core tactical decisions in RGB combat.

← [Back to README](#)

Damage Model

The RGB System uses a **layered damage model** that combines impact sources, penetration, armor, shields, and special abilities. This model keeps combat simple while still allowing tactical decisions.

This document explains how damage flows through the system.

Damage Resolution Flow

Damage in the RGB System follows a clear sequence:

```
Hit or Contact Check
  ↓
Impact Source
  ↓
Penetration
  ↓
Armor Reduction
  ↓
Shield Absorption
  ↓
Remaining Damage → Character
```

This flow shows how different elements of the system interact during combat.

Step-by-Step Damage Resolution

1. Attack Check

The attacker attempts to hit the defender.

Combat rules determine if the attack succeeds.

See:

- ../combat/attack_and_defense.md

If the attack fails, no damage is applied.

2. Impact Source

If the attack succeeds, identify the **impact source**.

Impact may come from a weapon, attribute, ability, procedure, or explicit exception. The default is:

- melee impact uses weapon value plus the attacker's **R** when the weapon or procedure permits it;
- firearm impact uses weapon value by default;
- explosive impact uses the explosive profile;
- ability impact uses the ability's declared effect.

Examples cannot override the declared impact source.

See:

- ../weapons/categories/firearms.md
- ../weapons/categories/melee.md
- ../weapons/categories/explosives.md

3. Penetration

Apply penetration to armor before armor reduces damage.

$$\text{Effective Armor} = \text{Armor} - \text{Penetration}$$

If effective armor is zero or less, armor does not reduce that hit.

See:

- ../weapons/mechanics/penetration.md

4. Armor Reduction

Armor provides **physical protection**.

Armor reduces incoming damage per hit after penetration.

Typical armor categories include:

- Light Armor
- Medium Armor
- Heavy Armor

See:

- ../equipment/armor.md

5. Shield Absorption

Some characters may possess shields.

Shields absorb remaining damage **after armor reduction** and before it reaches health or a state consequence.

Energy shields are typically calculated as:

$$\text{Shield} = B \times 3$$

See:

- ../equipment/shields.md

6. Remaining Damage

After armor and shields are resolved, the remaining damage is applied to the character.

This damage affects the character's health or creates a declared state consequence.

Special Abilities

Some abilities may modify how damage is applied.

Examples include:

- absorption techniques
- energy shields
- defensive martial techniques

See:

- ../core/skills_and_abilities.md

Damage Interaction Overview

The RGB damage model can be summarized as:

```
Impact Source
  ↓
Penetration
  ↓
Armor Reduction
  ↓
Shield Absorption
  ↓
Remaining Damage
```

This layered model keeps combat easy to understand while allowing different equipment and abilities to interact in meaningful ways.

RGB Interaction Model

The system can also be visualized as an interaction between the RGB vectors:

G
mobility / reaction

R - B
power / damage shield / energy

Each vector represents a different defensive or offensive strategy in combat.

- **R (Red)** changes the source of pressure through force, impact, or interruption.
- **G (Green)** changes relation to pressure through movement, timing, and positioning.
- **B (Blue)** preserves continuity through blocks, shields, stabilization, and resistance.

Together they create a balanced tactical system.

← [Back to README](#)

Movement

Movement in the **RGB System** is primarily governed by the **Green (G) vector**, which represents agility, mobility, and reaction speed.

The movement rules connect directly to the **RGB Tactical Triangle**, where each vector represents a different combat strategy.

G
mobility / positioning

R ----- B
power / damage shield / defense

- **R (Red)** focuses on damage and offensive power
- **G (Green)** focuses on mobility and positioning
- **B (Blue)** focuses on defense and durability

Movement therefore represents the **positional advantage** component of the RGB system.

Movement Distance

The distance a character can move during a single combat turn is:

Movement = $G \times 2$ meters

Example:

$G = 3$
Movement = 6 meters per turn

Higher **G** values allow characters to reposition faster and control the battlefield.

Tactical Grid

The RGB System supports tactical combat using a square grid.

Standard grid:

1 square = 1 meter

This allows precise positioning for:

- cover
- flanking
- line of sight
- area effects

Aerial Movement

If a character can fly, aerial movement uses the same formula:

Aerial Movement = $G \times 2$ meters

The Game Master may modify this depending on the setting.

Examples include:

- jetpacks
- magical flight
- superpowers

Combat Turn Structure

Each combat turn normally includes:

- **Movement** — a free positional allowance ($G \times 2$ meters); it does not consume the Action or Minor Action below.
- **Action**
- **Minor Action**

Examples of **Actions**:

- attack
- defend
- use ability

Examples of **Minor Actions**:

- reload weapon
- small reposition
- interact with environment

Terrain Effects

Terrain may reduce movement.

Terrain	Effect
-----Light	-1 movement
Moderate	movement ÷2
Difficult	movement ÷3

Terrain penalties apply **after calculating base movement**.

Example:

```
G = 4
Base Movement = 8 meters

Moderate terrain → 4 meters
```

Optional Rule: Sprint

Characters may perform a sprint to move faster.

```
Sprint = G × 3 meters
```

Restrictions:

- No attack during the same turn
- Only movement and minor actions allowed

This rule allows high mobility characters to reposition quickly.

Tactical Interpretation

The RGB vectors influence combat decisions:

```
Press → R
Reposition → G
Sustain → B
```

This creates three distinct combat strategies:

```
| Strategy | Vector |
|-----|-----|
Heavy attacker | R |
Mobile skirmisher | G |
Defensive guardian | B |
```

Movement therefore represents **tactical control of space** in the RGB combat system.

← [Back to README](#)

Equipment

This section describes the equipment used in the RGB System.

Equipment affects combat survivability, mobility, and tactical options.

Sections

Armor

Rules for armor types and their impact on protection and mobility.

→ [Armor](#)

Shields

Physical and energy shields that absorb damage.

→ [Shields](#)

Gear

General equipment and items that characters may carry.

→ [Gear](#)

← [Back to README](#)

Armor

Armor represents **physical protection** used to reduce incoming damage in the RGB System.

Armor interacts with the **Damage Model** as the per-hit reduction layer. Energy shields and other Blue (B) effects are separate absorption or preservation layers.

Armor Calculation

Armor protection is a fixed reduction value:

$$\text{Effective Armor} = \text{Armor} - \text{Penetration}$$

For example:

```
Armor = 4
Penetration = 1
Effective Armor = 3
```

Armor reduces damage per attack. Shields absorb remaining damage after armor.

Armor Types

Type	Protection	Mobility Penalty
-----	-----	-----
Light	2	-1 G
Medium	4	-2 G
Heavy	6	-3 G

Light Armor

- Minimal protection
- Low mobility penalty
- Suitable for agile characters

Medium Armor

- Balanced protection

- Moderate mobility reduction

Heavy Armor

- High physical protection
- Significant mobility penalty

Heavy armor users rely more on **R (strength)** and **B (defense)** than on mobility.

Armor Interaction with the RGB System

Armor interacts with the RGB vectors in the following way:

R → contributes pressure and physical impact
G → affected by armor mobility penalties
B → may provide separate preservation or shield effects

This relationship maintains the RGB tactical balance:

R → deal damage
G → avoid damage
B → absorb damage

Interaction with the Damage Engine

Armor reduces damage **after weapon penetration is applied**.

The simplified damage flow:

```
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

For full rules see:

- [Damage Model](#)

Design Philosophy

Armor in the RGB system follows three principles:

- **Simple values** – easy to calculate during combat
- **Tactical trade-offs** – higher protection reduces mobility
- **Compatibility with shields** – allows layered defenses

This ensures armor remains relevant without overpowering mobility or defensive builds.

See Also

- [Shields](#)
- [Damage Model](#)

← [Back to README](#)

Equipment

Equipment represents **tools, devices, and tactical items** that characters may carry or use during gameplay.

Unlike weapons or armor, general equipment usually **does not directly modify damage calculations**, but it can provide tactical advantages during exploration, investigation, or combat.

Equipment Categories

Equipment in the RGB System can generally be grouped into three categories.

Tools

Items used to interact with the environment or solve problems.

Examples:

- lockpicks
- repair kits
- medical kits
- climbing gear

These items typically allow characters to perform actions that would otherwise be impossible.

Devices

Technological or specialized equipment that provides functional advantages.

Examples:

- scanners
- communication devices
- surveillance tools
- drones

Devices may grant situational bonuses or new abilities depending on the campaign setting.

Tactical Equipment

Items designed to provide advantages during encounters.

Examples:

- flashlights
- smoke grenades
- signal beacons
- breaching tools

These items often influence **positioning, visibility, or battlefield control**.

Interaction with the RGB System

Equipment may interact with RGB vectors depending on the item.

Examples:

R → tools that enhance strength or force
G → mobility tools or positioning devices
B → energy devices or protective equipment

However, most general equipment **does not modify core vector values directly** unless specified by the Game Master.

Design Philosophy

Equipment in the RGB system follows three design principles:

- **utility over complexity** – equipment should enable actions rather than introduce heavy mechanics
- **tactical flexibility** – items should allow creative solutions
- **setting adaptability** – equipment may vary depending on the campaign world

For example:

- modern settings may include electronics and tactical gear
- fantasy settings may include magical tools
- science-fiction settings may include advanced technology

Relationship with Other Systems

Equipment interacts with several RGB modules:

Equipment → supports gameplay actions
Weapons → deal damage
Armor → reduces damage
Shields → absorb damage

Because of this separation, equipment remains **lightweight and modular** within the RGB system.

See Also

- [Armor](#)
- [Shields](#)
- [Firearms](#)

← [Back to README](#)

Shields

Shields are **defensive equipment or energy fields** that preserve a character under pressure.

Shields work together with **Armor** and the **Damage Model**, creating layered defenses.

There are two main types of shields:

- Physical Shields
- Energy / Magical Shields

Physical Shields

Physical shields represent defensive equipment carried by a character.

They provide additional protection but may impose mobility penalties.

Physical shields may be **destroyed, dropped, or lost** depending on the Game Master's decision.

Shield Type	Protection	Mobility Penalty
-----	-----	-----
Light Shield	+1	0
Medium Shield	+2	-1 G
Heavy Shield	+3	-2 G

Characteristics

- Enables block, cover, or protect-ally procedures
- May affect mobility (G)
- Subject to equipment loss or damage

Physical shields are most commonly used in:

- medieval settings
- tactical combat environments
- close-quarters combat

Energy / Magical Shields

Energy or magical shields represent **protective energy fields**, advanced technology, or magical barriers.

Unlike armor, these shields **absorb damage rather than reducing it**.

Characteristics:

- do not regenerate during combat
- have a fixed value during a fight
- return to maximum after the combat ends

Shield value is determined by the Blue vector.

$$\text{Shield} = B \times 3$$

Example:

$$\begin{aligned} B &= 3 \\ \text{Shield} &= 9 \end{aligned}$$

Energy shields represent the **absorption layer** in the RGB damage system and are a typed expression of B as preservation.

Interaction with the RGB Damage Model

Shields are applied **after armor reduction**.

Damage resolution order:

```
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

This layered defense system keeps combat simple while preserving tactical depth.

Interaction with RGB Vectors

Shields interact with the three RGB vectors in different ways.

R → contributes pressure and impact
G → may be reduced by heavy shields
B → determines energy shield capacity

This maintains the RGB combat philosophy:

R → deal damage
G → avoid damage
B → absorb damage

Design Philosophy

Shields in the RGB system follow three principles:

- **layered defense** – armor reduces damage, shields absorb damage
- **tactical trade-offs** – heavier shields reduce mobility
- **modular design** – shields adapt to different settings

Examples:

- medieval campaigns → physical shields
- science fiction → energy shields
- fantasy settings → magical shields

See Also

- [Armor](#)
- [Damage Model](#)
- [Attack and Defense](#)

← [Back to README](#)

Weapons

Weapons define the primary methods of dealing damage in the RGB System.

The system separates **weapon categories** from **weapon mechanics**.

Weapon Categories

These pages describe the different weapon types available in the system.

- [Firearms](#)
- [Melee Weapons](#)
- [Explosives](#)

Weapon Mechanics

These rules are shared across weapons and define how they behave in combat.

- [Weapon Ranges](#)
- [Penetration](#)

See Also

Other parts of the system that interact directly with weapons:

- [Damage Model](#)
- [Armor](#)
- [Attack and Defense](#)

← [Back to README](#)

Reference

This section contains reference materials and practical examples for the **RGB System**.

These documents help players and game masters understand how the rules work in practice.

Examples

Character Sheet

Standard character sheet template for RGB characters.

→ [Character Sheet](#)

Combat Example

Combat example using the system's rules.

→ [Combat Example](#) → [Combat Walkthrough](#)

Gameplay Example

Narrative example of the system in use during a session.

→ [Gameplay Example](#)

Gameplay Loop

Explains the typical flow of an RGB session.

→ [Gameplay Loop](#)

Design Notes

RGB Interaction Model

Explains the relationship between vectors, damage, and combat.

→ [RGB Interaction Model](#)

System Architecture

Observations on the RGB System's architecture.

→ [RGB System Architecture](#)

Extra Notes

Additional observations about the system's design.

→ [Extra Notes](#)

Known Gaps / Maintenance Notes

- **Link checking:** this documentation is currently checked manually. Adding an automated mark-down link-checker to CI is recommended to prevent broken cross-references from going unnoticed.
- **RGB Ability Engine:** an earlier reference to a `rgb_ability_engine.md` document was removed from `rgb_system_engine.md` — no such document was ever authored in this repository.

← [Back to README](#)

Character Sheet

The character sheet summarizes the **core values of a character in the RGB System**.

It organizes the three RGB vectors together with derived values and equipment.

Basic Information

Name
Level

RGB Vectors

R (Red) → Pressure / Impact
G (Green) → Relation / Reaction
B (Blue) → Preservation / Energy

These vectors determine most actions in the system.

Derived Values

Derived values are calculated from RGB vectors.

Health = 4 + R + B

Optional energy shield (if the setting allows it):

Energy Shield = B × 3

Optional ability fatigue system:

Callback (if used in the campaign setting)

Equipment

Equipment
Weapons
Armor

Equipment interacts with the combat and damage systems.

Relationship with the RGB System

The character sheet connects several modules of the RGB system.

Vectors → define character capability
Health → derived from R + B
Shield → derived from B
Equipment → modifies combat interaction

Design Philosophy

The RGB character sheet follows three design principles:

- **minimal structure** – few numbers define the character
- **fast readability** – all core values visible at a glance
- **system modularity** – optional systems (shield, callback) can be added depending on the campaign

This allows the same sheet to work in:

- modern campaigns
- fantasy settings
- science-fiction worlds
- superhero games

Minimal Character Sheet Example

Name: Stormy
Level: 3

R: 3
G: 3
B: 1

Health: 8
Shield: 3

Armor: Medium Armor
Weapons: Dual Pistols
Equipment: Tactical Gear

← [Back to README](#)

Combat Example

This example demonstrates how combat is resolved using the **RGB System**.

Two characters engage in combat and resolve actions using the core rules:

- RGB vectors
- movement
- attack and defense
- armor and shields
- damage calculation

Characters

Character A – Assault Fighter

R = 3
G = 2
B = 1

Derived values:

Health = $4 + R + B = 8$
Shield = $B \times 3 = 3$
Movement = $G \times 2 = 4$ meters

Equipment:

- Medium Armor (Protection 4)
- Rifle

Character B – Mobile Operator

R = 2
G = 3
B = 2

Derived values:

```
Health = 8  
Shield = 6  
Movement = 6 meters
```

Equipment:

- Light Armor (Protection 2)
- SMG

Combat Turn Example

Step 1 — Positioning

Characters move according to:

```
Movement = G × 2 meters
```

Character B moves first due to higher mobility.

Step 2 — Attack

Character A fires the rifle.

Example weapon damage:

```
Impact Source = rifle  
Weapon Damage = 6  
Penetration = 2
```

Step 3 — Armor Interaction

Armor reduces incoming damage after penetration.

Example:

```
Target Armor = 2  
Penetration = 2  
Effective Armor = 0
```

Armor is bypassed.

Step 4 — Shield Absorption

Shield absorbs damage before health.

```
Shield = 6  
Damage = 6
```

Result:

```
Remaining Shield = 0  
Health Damage = 0  
Remaining Health = 8
```

Second Attack

Character B attacks with an SMG.

```
Weapon Damage = 4  
Penetration = 1
```

Step 5 — Armor Interaction

```
Target Armor = 4  
Penetration = 1  
Effective Armor = 3
```

```
Damage = 4  
Effective Armor = 3  
Remaining Damage = 4 - 3 = 1
```

Step 6 — Shield Absorption

Character A still has full shield (3), so the remaining damage hits shield before health.

```
Shield = 3  
Remaining Damage = 1
```

Result:

```
Remaining Shield = 3 - 1 = 2  
Health Damage = 0  
Remaining Health = 8
```

Damage Flow Summary

The RGB system resolves damage using a layered structure.

```
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

Tactical Interpretation

The example shows the role of the RGB vectors:

```
R → determines pressure and impact
G → determines relation, mobility, and positioning
B → determines preservation and shield capacity
```

Each vector supports a different combat strategy.

Design Goal

Combat resolution in the RGB system is designed to be:

- fast to calculate
- tactically meaningful
- consistent across settings

Because the same formulas apply to all characters, the system remains easy to balance.

See Also

- [Attack and Defense](#)
- [Movement](#)
- [Damage Model](#)

← [Back to README](#)

Combat Walkthrough

This example demonstrates a **complete combat encounter** using the RGB System.

Two characters engage in combat inside a tactical environment.

Characters

Stormy

R = 3

G = 3

B = 2

Derived values:

Health = $4 + R + B = 9$

Shield = $B \times 3 = 6$

Movement = $G \times 2 = 6$ meters

Equipment:

- Medium Armor
- Dual Pistols

Security Agent

R = 2

G = 2

B = 1

Derived values:

Health = 8

Shield = 3

Movement = 4 meters

Equipment:

- Light Armor
- SMG

Initial Situation

The characters begin **6 meters apart**.

Stormy has partial cover behind a vehicle.

Turn 1

Movement

Stormy moves 2 meters to improve line of sight.

Movement formula:

$$\text{Movement} = G \times 2$$

Stormy could move up to **6 meters**, but chooses a smaller reposition.

Attack

Stormy fires one pistol.

Weapon damage:

$$\text{Damage} = 4$$

$$\text{Penetration} = 1$$

Armor Interaction

Target armor:

$$\text{Light Armor} = 2$$

$$\text{Penetration} = 1$$

$$\text{Effective Armor} = 1$$

Damage after armor:

$$4 - 1 = 3$$

Shield Interaction

Target shield:

Shield = 3

Shield absorbs all damage.

Remaining Shield = 0

Health Damage = 0

Turn 2

The security agent retaliates.

Weapon:

SMG Burst

Damage = 5

Penetration = 1

Stormy armor:

Medium Armor = 4

Effective Armor = 3

Damage:

$5 - 3 = 2$

Stormy shield absorbs damage.

Remaining Shield = 4

Health Damage = 0

Combat Summary

This walkthrough illustrates the RGB combat model.

Damage flow:

Weapon

↓

Penetration

↓

Armor

↓

Shield

↓

Character

Tactical Observation

Stormy benefits from:

- higher mobility (G)
- stronger shield (B)

The agent relies more on weapon damage.

This shows how **different RGB builds influence combat strategy**.

← [Back to README](#)

Mathematical Balance Observation in the RGB System

This document describes an **emergent property** of the RGB System that appears when the main system formulas are analyzed together.

Even though the system was not explicitly designed for this purpose, the RGB mechanics produce a **natural balance between the three vectors: R, G, and B.**

Core RGB Formulas

The core rules define the following relationships.

Health

$$\text{Health} = 4 + R + B$$

Movement

$$\text{Movement} = G \times 2$$

Energy Shield

$$\text{Shield} = B \times 3$$

Special Absorption

Special absorption allows characters to convert vector points into damage reduction.

R → physical impact pressure
B → energy or magical absorption

Vector Influence in Combat

Each RGB vector influences combat in a different way.

Vector	Combat Role
-----	-----
R	Pressure and physical impact
G	Mobility, evasion and positioning
B	Energy shield and special defenses

Resistance Scaling Example

Consider a character with **5 points in each vector**.

Health = 4 + 5 + 5 = 14
Shield = 5 × 3 = 15

Observation:

Shield ≈ Health

This indicates that **energy protection and physical resilience reach similar levels of durability**, but through different mechanics.

Balance Between R and B

Both vectors can provide survivability.

High R Character

Advantages:

- higher pressure and impact
- contribution to health through physical pressure tolerance

High B Character

Advantages:

- stronger energy shield
- protection against special damage

Result:

Both builds can reach similar levels of overall resistance using different systems.

Role of the G Vector

The Green vector does not increase raw durability.

Instead, it improves:

- evasion
- movement
- tactical positioning

This creates a third survival strategy:

Avoid damage instead of absorbing it

Vector Multipliers

Vector	Multiplier
-----	-----
R	+1 health and pressure
G	×2 (movement)
B	×3 (shield)

These multipliers create three defensive approaches:

R → pressure tolerance and impact
B → preservation through health contribution and shield
G → avoidance through mobility

Emergent Character Archetypes

Without defining explicit classes, the system naturally produces character styles.

Style	Dominant Vector	
-----Physical tank		High R
Evasive fighter	High G	
Energy defender	High B	

Hybrid Builds

Characters can also combine vectors effectively.

Example:

```
R = 6
B = 6
```

Derived values:

```
Health = 16
Shield = 18
```

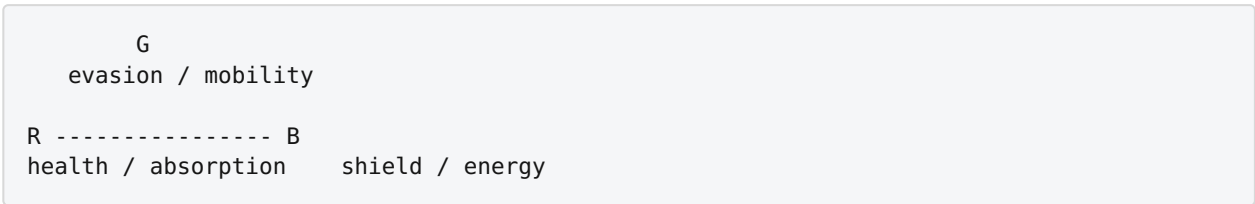
Approximate total resistance:

```
16 + 18 = 34
```

This demonstrates that **hybrid builds remain competitive** within the RGB system.

The RGB Balance Triangle

The system can be visualized as a functional triangle.



Each vector represents a different strategy for survival.

Conclusion

The RGB System demonstrates a **structural balance between its three core vectors**.

- **R** increases direct resilience through health
- **B** increases external protection through shields
- **G** improves survival by avoiding damage

Together these mechanics create a naturally balanced system without requiring rigid classes or complex balancing rules.

← [Back to README](#)

Gameplay Example

This example illustrates how a **typical session flows in the RGB System**.

Gameplay usually alternates between three main phases:

- narrative interaction
- exploration
- combat

These phases form the **core gameplay loop** used in most RGB campaigns.

Example Scenario

A small team of operatives is investigating suspicious activity in an industrial district.

The group consists of:

- **Stormy** – mobile combat specialist
- **Security Agent** – trained tactical operator
- **Support Technician** – equipment and analysis specialist

The Game Master describes the environment and the players decide how to proceed.

Phase 1 — Narrative

The session begins with role-playing and narrative interaction.

Example:

The team receives information about illegal technology shipments in a warehouse district.

Players may:

- question NPCs
- gather information
- plan their approach
- negotiate or intimidate

No strict mechanical rules are required in this phase unless the Game Master requests tests.

Phase 2 — Exploration

Players move through the environment and interact with objects or locations.

Examples:

- searching buildings
- bypassing security systems
- scanning the environment
- avoiding patrols

The Game Master may request RGB checks when appropriate.

Example:

```
Agility Check → based on G  
Strength Check → based on R  
Technical / Energy interaction → based on B
```

Exploration encourages creative use of equipment and abilities.

Phase 3 — Combat

If a conflict occurs, the game enters the **combat phase**.

Combat uses the RGB combat system.

Basic sequence:

```
Movement  
↓  
Action (attack / ability)  
↓  
Defense resolution  
↓  
Damage calculation
```

Example encounter:

Stormy encounters two hostile agents inside the warehouse.

Players use:

- movement (G)
- attacks (R)
- shields or abilities (B)

The encounter is resolved using the RGB combat rules.

Session Resolution

After the encounter, the game returns to narrative play.

Possible outcomes:

- characters recover from combat
- players investigate the location
- new story elements emerge

This cycle continues throughout the session.

RGB Gameplay Loop

The overall gameplay structure can be summarized as:

```
Narrative
↓
Exploration
↓
Combat
↓
Recovery / Story Progression
```

This loop allows the RGB system to support different play styles while keeping rules simple.

Design Philosophy

The RGB system separates **narrative flow** from **combat mechanics**.

- narrative phases remain flexible
- exploration encourages player creativity
- combat uses clear tactical rules

This balance allows the same system to work in:

- modern campaigns
- fantasy worlds
- science-fiction settings
- superhero stories

See Also

- [gameplay_loop.md](#)
- [combat_example.md](#)
- [combat_walkthrough.md](#)

← [Back to README](#)

Gameplay Loop

The RGB System follows a simple gameplay loop that connects character creation, exploration, and combat.

This loop describes how the system is typically used during play.

1. Character Creation

Players create characters using the RGB vector system.

Characters distribute points between:

- **R (Red)** — power, attacks and physical strength
- **G (Green)** — agility, mobility and reaction
- **B (Blue)** — shields, energy and special resistance

Additional abilities and equipment may be selected depending on the campaign.

→ See [Character Creation](#)

2. Exploration and Interaction

During gameplay, characters interact with the environment using their RGB attributes.

Typical actions include:

- investigation
- movement
- social interaction
- environmental challenges

The game master determines how RGB attributes influence these actions.

3. Tactical Positioning

Before combat begins, characters position themselves in the environment.

The RGB System supports tactical combat using grid-based positioning.

Typical considerations include:

- distance between characters
- line of sight
- cover and terrain
- movement speed

→ See [Movement](#)

4. Combat Resolution

Combat is resolved using the RGB combat rules.

Combat interactions involve:

- attack and defense comparisons
- weapon damage
- armor and shield interaction
- tactical movement

→ See [Attack and Defense](#)

5. Damage and Consequences

Damage affects characters through:

- armor
- shields
- vitality

Special abilities may modify these interactions.

→ See [Weapons](#)

6. Recovery

After encounters, characters may recover depending on the rules defined by the game master.

Recovery may include:

- health recovery
- ability cooldowns
- equipment repair

Gameplay Cycle

A typical session follows this structure:

Character Creation

- Exploration
- Tactical Positioning
- Combat
- Consequences
- Recovery

This cycle repeats as the story progresses.

← [Back to README](#)

RGB System – Damage Model and Interaction Design Notes

This document consolidates the design observations about the RGB System damage model, its mathematical balance, and how the RGB vectors interact with combat mechanics.

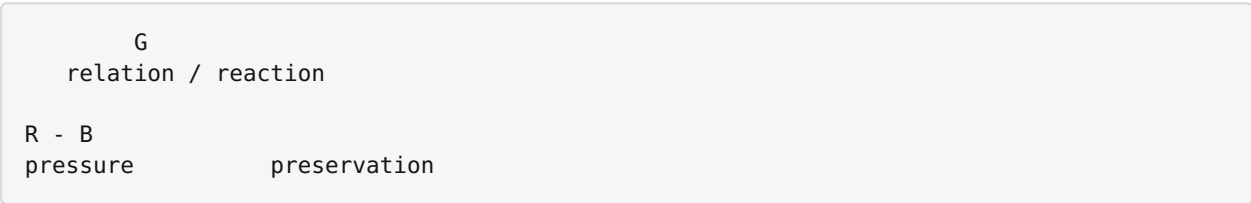
The goal of this document is to explain **why the system behaves consistently and remains balanced**, while keeping the rules simple.

RGB Interaction Model

The RGB System is built around three fundamental vectors:

- **R (Red)** — pressure, impact, disruption and physical force
- **G (Green)** — relation, mobility, timing and reaction
- **B (Blue)** — preservation, shields, energy and special resistance

These vectors interact to create tactical gameplay.



Each vector influences a different aspect of gameplay:

Vector	Gameplay Role
R	Pressure, impact and physical force
G	Mobility, positioning and evasion
B	Preservation, energy shields and special defenses

System Interaction Flow

The RGB System connects character attributes, equipment and combat mechanics.

```
RGB Vectors
↓
Character Creation
↓
Equipment Selection
↓
Combat Interaction
↓
Damage Resolution
```

This modular structure allows the system to adapt to multiple settings such as:

- modern campaigns
- fantasy worlds
- science fiction settings
- super-powered universes

RGB Damage Model

Combat damage in the RGB System follows a layered structure.

```
Hit or Contact Check
↓
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

This layered approach ensures that weapons, armor and shields interact predictably.

Core System Formulas

The RGB system defines character durability using simple formulas.

```
Health = 4 + R + B
Shield = B × 3
```

Weapons, attributes, abilities or procedures declare the **impact source**, while armor provides **fixed reduction values**.

Penetration modifies the effective armor value.

Effective Armor

Penetration interacts with armor through a simple relationship:

$$\text{Effective Armor} = \text{Armor} - \text{Penetration}$$

Final damage becomes:

$$\text{Damage After Armor} = \text{Impact Source} - \text{Effective Armor}$$

This keeps calculations simple and predictable.

Natural Damage Scale

If the following ranges are used:

Element	Typical Scale
Weapons	3–10
Armor	1–6
Penetration	0–4

A natural balance relationship appears:

$$\text{Impact Source} \approx \text{Armor} + \text{Penetration}$$

This ensures that weapons and defenses remain balanced.

Example

Light Weapon

$$\begin{aligned} \text{Damage} &= 4 \\ \text{Penetration} &= 1 \\ \text{Armor} &= 4 \end{aligned}$$

$$\begin{aligned} \text{Effective Armor} &= 4 - 1 = 3 \\ \text{Final Damage} &= 4 - 3 = 1 \end{aligned}$$

Heavy Weapon

Damage = 8
Penetration = 3
Armor = 6

Effective Armor = $6 - 3 = 3$
Final Damage = $8 - 3 = 5$

Shield Interaction

After armor reduction, shields absorb remaining damage.

Remaining Damage – Shield

Shields follow the RGB formula:

Shield = $B \times 3$

This creates two defensive layers:

Defense Type	Function
Armor	Reduces damage per attack
Shield	Absorbs accumulated damage

This separation simplifies combat resolution.

Balance Rule of Thumb

When designing weapons and armor, the following approximation keeps combat balanced:

Armor $\approx$ Impact Source / 2
Penetration $\approx$ Impact Source / 3

This keeps most attacks meaningful while preserving defensive value.

Unexpected Advantage: Multiple Enemy Combat

One interesting property of the RGB damage model is that it performs well when characters face **multiple opponents simultaneously**.

Many RPG systems struggle with this scenario because defensive values scale poorly against repeated attacks.

In RGB, the layered model distributes damage naturally:

```
Impact Source → Penetration → Armor → Shield → Character
```

Armor

Armor reduces damage **for each individual attack**, preventing weak enemies from overwhelming armored characters too quickly.

Shield

Shields absorb **total accumulated damage**, acting as a buffer against multiple small attacks.

Tactical Consequences

This creates a stable combat dynamic.

```
| Enemy Type | Result |  
|--|  
Many weak enemies | Armor reduces most attacks |  
Few strong enemies | Penetration becomes important |  
Energy-heavy enemies | Shields become critical |
```

Because armor works per hit and shields work cumulatively, the system remains stable even with many attackers.

Design Conclusion

The RGB damage structure forms a clear defensive hierarchy:

```
Weapon  
↓  
Penetration  
↓  
Armor  
↓  
Shield  
↓  
Character
```

This structure:

- keeps calculations simple
- supports tactical combat

- scales naturally with equipment
- handles multi-enemy encounters well

The interaction between **R, G and B vectors** ensures that different character builds remain viable while encouraging tactical choices during combat.

← [Back to README](#)

RGB System Architecture – Layered Design Observation

This note documents an interesting structural property that appears when the RGB documentation is analyzed as a whole.

Although the RGB System was designed as a simple tabletop RPG system, the documentation naturally reveals a **layered architecture** similar to what is often found in game engines.

This structure improves clarity, modularity and long-term expandability.

Layered Architecture of the RGB System

When the documentation is organized by function, the system forms the following layers:

```
System Layer
↓
Gameplay Layer
↓
Combat Layer
↓
Damage Layer
↓
Equipment Layer
```

Each layer answers a different design question.

1. System Layer

Documents:

- [System Overview](#)
- [RGB Damage Interaction Model](#)

Purpose:

This layer explains the **core philosophy and structure of the system**.

It defines the RGB vectors:

- **R (Red)** – damage and physical power
- **G (Green)** – mobility and reaction
- **B (Blue)** – shields and energy defense

It also shows how the system modules connect.

Equivalent concept in game engines:

Core system design.

2. Gameplay Layer

Documents:

- [Gameplay Loop](#)
- [Combat Decision Model](#)

Purpose:

This layer explains **how players interact with the system during play**.

Typical flow:

~~~~~text Character Creation ↓ Exploration ↓ Combat ↓ Recovery

During combat the player usually chooses between:

```
```text
Press
Reposition
Sustain
```

Which map naturally to:

```
Press → R
Reposition → G
Sustain → B
```

Equivalent concept in game engines:

Gameplay framework.

# 3. Combat Layer

Documents:

- [Attack and Defense](#)
- [Movement](#)

Purpose:

Defines how characters interact during encounters.

This includes:

- movement rules
- attack resolution
- defensive reactions
- positioning

Equivalent concept in game engines:

Combat system.

# 4. Damage Layer

Documents:

- [Damage Model](#)
- [RGB Damage Interaction Model](#)

Purpose:

Handles the internal logic of damage calculation.

The RGB damage engine follows this structure:

```
Weapon
↓
Penetration
↓
Armor
↓
Shield
↓
Character
```

This layer allows combat to remain tactical while keeping calculations simple.

Equivalent concept in game engines:

Damage engine.

# 5. Equipment Layer

Documents:

- [Armor](#)
- [Shields](#)
- [Firearms](#)
- [Melee](#)
- [Explosives](#)

Purpose:

Defines the **data values used by the combat and damage systems.**

Examples:

- weapon damage
- armor reduction
- penetration values
- shield capacity

Equivalent concept in game engines:

Data layer.

# Complete RGB System Architecture

When combined, the RGB documentation forms the following structure:

```
graph TD; SD[System Design] --> SO[System Overview]; SD --> RIM[RGB Interaction Model]; SD --> GP[Gameplay]; SD --> C[Combat]; SD --> DE[Damage Engine]; SD --> ED[Equipment Data]; GP --> GL[Gameplay Loop]; GP --> CDM[Combat Decision Model]; C --> M[Movement]; C --> AD[Attack & Defense]; DE --> DM[Damage Model]; ED --> A[Armor]; ED --> S[Shields]; ED --> W[Weapons];
```

- System Design
  - System Overview
  - RGB Interaction Model
- Gameplay
  - Gameplay Loop
  - Combat Decision Model
- Combat
  - Movement
  - Attack & Defense
- Damage Engine
  - Damage Model
- Equipment Data
  - Armor
  - Shields
  - Weapons

# Design Consequences

This layered architecture produces several advantages.

## Modularity

New subsystems can be added without modifying core rules.

Examples:

- magic systems
- cybernetics
- vehicles
- superpowers

## Easier Balance

Most balancing occurs in:

Damage Layer  
Equipment Layer

The rest of the system remains stable.

## Clear Documentation

Each layer answers a different question:

| Layer     | Question                           |
|-----------|------------------------------------|
| System    | How the system is structured       |
| Gameplay  | How players interact with the game |
| Combat    | How characters interact            |
| Damage    | How damage is calculated           |
| Equipment | Which numerical values exist       |

# Final Observation

The RGB System behaves less like a traditional RPG rule set and more like a **reusable RPG system engine**.

This makes the system particularly well suited for:

- modern campaigns
- fantasy worlds
- science fiction settings
- super-powered universes

In this model:

RGB System → rule engine  
Setting (e.g., Aurora) → world built on top of the engine

This separation allows the RGB System to remain generic while different settings provide narrative context.

← [Back to README](#)

# RGB System Engine

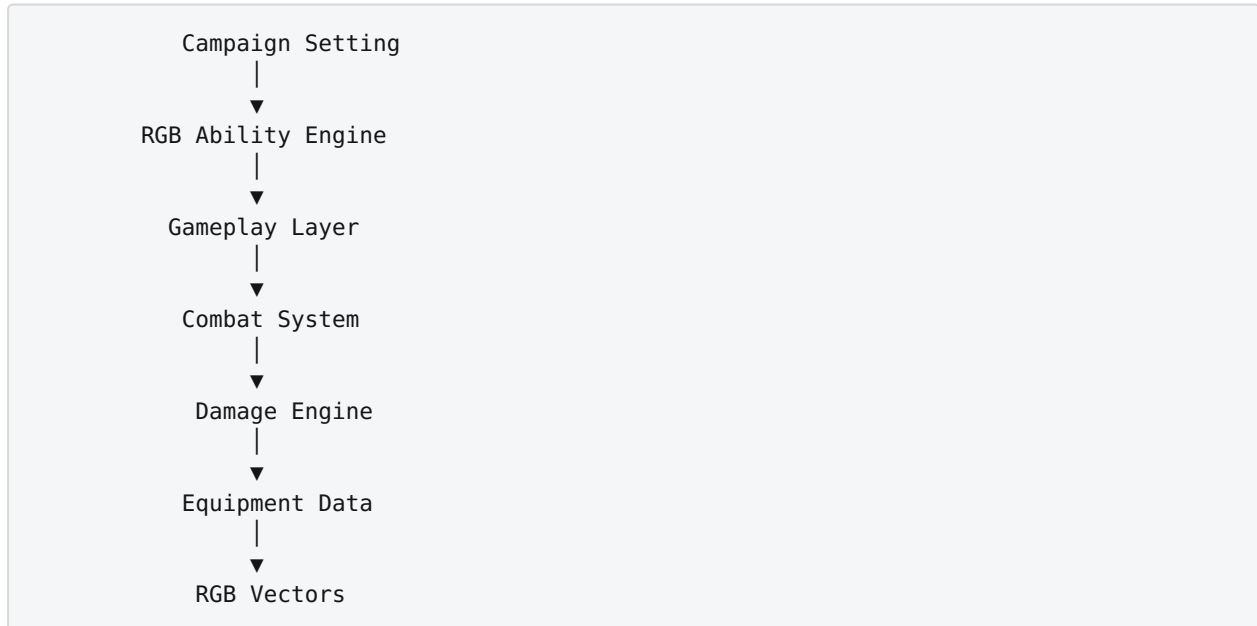
The **RGB System Engine** describes the full architecture of the RGB role-playing system. It connects all system components into a layered structure similar to a game engine.

This model helps explain how the RGB system remains:

- modular
- scalable
- easy to balance
- adaptable to many settings

## RGB System Architecture

The RGB system can be understood as a layered engine:



Each layer builds on the one below it.

## Layer Description

### RGB Vectors (Core Layer)

The entire system is built on the three RGB vectors.

```
      G
  mobility / positioning

R ----- B
power / damage      shield / defense
```

Vectors define the core mechanics:

- **R (Red)** → offensive power
- **G (Green)** → mobility and reaction
- **B (Blue)** → defense and energy

All other systems derive from these three values.

### Equipment Data

This layer defines numerical values used by the combat system.

Examples:

- weapon damage
- armor reduction
- penetration values
- shield capacity

Documents:

- [Firearms](#)
- [Melee](#)
- [Armor](#)
- [Shields](#)

### Damage Engine

The damage engine determines how attacks affect characters.

```
Weapon
↓
Penetration
↓
Armor
↓
Shield
↓
Character
```

Documents:

- [Damage Model](#)
- [RGB Damage Interaction Model](#)

## Combat System

The combat layer defines interaction between characters.

Examples:

- attack resolution
- movement
- positioning
- defense reactions

Documents:

- [Movement](#)
- [Attack and Defense](#)

## Gameplay Layer

Defines how players interact with the system during play.

Typical gameplay loop:

```
Character Creation
↓
Exploration
↓
Combat
↓
Recovery
```

Documents:

- [Character Creation](#)
- [Gameplay Loop](#)

- [Combat Decision Model](#)

## RGB Ability Engine

This layer introduces abilities, powers, and special mechanics.

Abilities are built using the RGB vectors.

```
Vector
Cost
Effect
Duration
Limit
```

Documents:

- [Skills and Abilities](#)

## Campaign Setting

The top layer defines the narrative world.

Examples:

- modern campaigns
- fantasy worlds
- science fiction
- superhero universes

Example:

```
RGB System → rule engine
Aurora → campaign setting built on top of the engine
```

## Complete System Flow

The full interaction between layers:

```
graph TD; Vectors --> EquipmentData[Equipment Data]; EquipmentData --> DamageEngine[Damage Engine]; DamageEngine --> CombatSystem[Combat System]; CombatSystem --> GameplayRules[Gameplay Rules]; GameplayRules --> AbilityEngine[Ability Engine]; AbilityEngine --> CampaignSetting[Campaign Setting];
```

Vectors  
↓  
Equipment Data  
↓  
Damage Engine  
↓  
Combat System  
↓  
Gameplay Rules  
↓  
Ability Engine  
↓  
Campaign Setting

## Design Advantages

The RGB System Engine provides several advantages.

## Modular Design

New subsystems can be added without rewriting core rules.

Examples:

- magic systems
- vehicles
- cybernetics
- superpowers

## Easier Balance

Balance occurs primarily in:

Damage Engine  
Equipment Data

The rest of the system remains stable.

## Clear Documentation

Each layer answers a specific question.

| Layer         | Question                         |                           |
|---------------|----------------------------------|---------------------------|
| -----         | -----                            | -----Vectors              |
| Equipment     | What numbers exist?              | What defines a character? |
| Damage Engine | How damage works?                |                           |
| Combat System | How characters interact?         |                           |
| Gameplay      | How players interact?            |                           |
| Abilities     | What special powers exist?       |                           |
| Setting       | What world the story happens in? |                           |

## Final Observation

The RGB System behaves less like a traditional RPG ruleset and more like a **modular RPG engine**.  
Because of this architecture, the same core system can support many different genres while remaining simple and consistent.

← [Back to README](#)

# Glossary

Terms used throughout the RGB System rules, gathered in one place. This page is not normative on its own — each entry links to the rule document that actually defines the term; the glossary only points there.

## R, G, B

The three vectors the system is built around. See [Attributes](#).

- **R (Red)** — pressure and physical impact: attack, force, disruption.
- **G (Green)** — relation, timing, and positioning: movement, evasion, reach.
- **B (Blue)** — preservation under pressure: endurance, blocking, shields, stabilization.

## Margin

The difference between an attacker's effective **R** and a defender's effective **G** in an attack, used to classify the outcome (strong hit, hit, hit with cost, miss with opportunity, clear miss) instead of a flat hit/miss. See [Attack and Defense](#).

## Health

A character's vitality resource,  $\text{Health} = 4 + \text{R} + \text{B}$ . See [Character Creation](#).

## Impact

The pre-mitigation force of a hit, before penetration, armor reduction, or shield absorption are applied. See [Damage Model](#).

## Penetration

How much of a weapon's impact bypasses armor before armor reduction is applied. See [Penetration](#).

## Armor Reduction

The mitigation layer that reduces impact per hit, applied after penetration. See [Armor](#).

## Shield Absorption

The mitigation layer that absorbs remaining impact after armor reduction, drawn from a **B**-derived resource. See [Shields](#).

## State

A tagged condition a character can carry (e.g. **injured**, **suppressed**, **hidden**), created, changed, or cleared by specific procedures. Fifteen states exist across physical, positional, tactical, defensive, and informational categories.

## Procedure

A named defensive or tactical action with a stable identity and vector ownership: Evade, Reposition, Block, Interrupt, Counterpressure. See [Attack and Defense](#).

## Ability

A discrete character capability declared with a minimal governed contract (id, vector, tier, requirements, cost, range, duration, effect, limits, tags, source status). See [Skills and Abilities](#).

## Tier

An ability's power/complexity band, used alongside **requirements** to gate when a character can take it. See [Skills and Abilities](#).

# ADR-001: AI-First Documentation Authority

## Status

Accepted.

## Context

RGB Core V2 needs stable documentation identity before Maker, Specialist, tooling, Library, PDF, landing, bundles, AI context packs, or search indexes can consume rules safely.

The project uses Markdown as a human review surface, but AI-first retrieval, provenance, localized drift checks, projection manifests, and generation gates need stable source IDs and machine-readable contracts.

## Decision

Adopt a bridge-first documentation authority model:

- English Markdown under `docs/core/en/**` remains the canonical bridge for Core V2 rule text until a specific semantic source unit is promoted.
- PT-br Markdown under `docs/core/PT-br/**` remains a localized projection and cannot introduce mechanics not present in canonical English or canonical semantic source units.
- `docs/core/semantic/core-v2.index.json` provides stable IDs, authority type, source status, projection paths, relationships, retrieval metadata, and provenance.
- Accepted semantic source files under `docs/core/semantic/source/**` may become canonical for specific promoted units when they preserve Markdown EN/PT-br projection paths, source IDs, provenance, and validation coverage.
- Generated artifacts, indexes, context packs, Library pages, PDF output, landing summaries, and bundles are derived projections only.

## Consequences

- Consumers get stable IDs and provenance without waiting for a repository-wide rewrite.

- Tooling can validate source paths, projection paths, relationships, translations, consumer contracts, projection ownership, and generated output authority.
- Promoted semantic source files can replace Markdown as canonical rule authority for their specific source IDs.
- Generated artifacts remain non-canonical and must trace back to source IDs.

## Rejected Alternatives

- Big-bang semantic rewrite.
- Markdown-only long-term source model.
- Generated artifacts as rule authority.

# ADR-002: RGB Core V2 Design Control

## Status

Accepted.

## Context

RGB Core V2 needed a controlled design surface before downstream tracks such as Maker, Specialist, tooling, Library, PDF, landing, and bundles could consume the model safely.

## Decision

Treat Core V2 as the authoritative source of the playable RGB model. Core owns:

- vector grammar;
- procedures;
- resources;
- tactical states;
- ability contract boundaries;
- documentation authority for rules, procedures, examples, and design notes.

Downstream tracks consume Core decisions. They do not define Core mechanics.

## Consequences

- Core documentation can be normalized before downstream product surfaces.
- Maker and Specialist must consume stable Core IDs and contracts.
- Library, PDF, generated bundles, search indexes, and AI context packs remain derived surfaces.
- Documentation authority must stay explicit when rules are promoted into semantic source units.

# ADR-003: Documentation L10n Maintenance Pattern

## Status

Accepted.

## Context

RGB System keeps English documentation as the canonical technical source and Portuguese documentation as a localized reader-facing surface. Path parity is useful, but it does not show whether a localized document is current, stale, or missing after an English source changes.

Core V2 documentation updates exposed this maintenance cost: English rule documents changed first, and PT-br alignment required a separate review pass.

## Decision

Adopt a lightweight advisory L10n manifest for documentation maintenance.

The project path model is:

|                    |                                            |
|--------------------|--------------------------------------------|
| Canonical source   | docs/core/en/**                            |
| Localized surface  | docs/core/PT-br/**                         |
| Traceability layer | docs/core/semantic/l10n-manifest.v0.1.json |

The manifest records:

- canonical English source path;
- localized path;
- target locale;
- authority type;
- translation status;
- source revision marker;
- optional notes.

English remains the canonical technical source. PT-br documents are localized reader-facing projections and must not introduce mechanics that are absent from the English source.

The first manifest is advisory. It supports review and tooling, but it does not generate localized documentation and does not block publication by itself.

## Authority Types

- `concept`
- `rule`
- `procedure`
- `example`
- `design_note`
- `translation`
- `generated_artifact`

## Translation Statuses

- `current`
- `needs_review`
- `stale`
- `missing`
- `intentionally_divergent`
- `not_applicable`

## Review Workflow

1. Update the canonical English document.
2. Identify affected localized documents through the manifest.
3. Mark affected PT-br entries as `needs_review` or `stale`.
4. Update PT-br content in a separate documentation task.
5. Return the manifest entry to `current` only after review.

## Consequences

- Maintainers can see source-to-localized traceability without reading every pair manually.
- Future Library, PDF, bundle, search, and Specialist consumers can disclose localization status before relying on localized content.
- Full semantic-source migration, per-document frontmatter, and automatic translation remain separate future decisions.

# ADR-004: Modular Monolith In A Single Monorepo

## Status

Proposed.

## Context

RGB System V2 spans several planned products: Core rules, Maker, Specialist, Tooling, Library (HTML/PDF), and a landing page. The V2 initial analysis (base\_project intake, section 20) lists two viable repository strategies: a monorepo or separate repositories per product.

The current state favors consolidation:

- the products share one canonical source ( docs/ Markdown + YAML) and one contract vocabulary that is still in formation;
- the Go scaffold already models products as component boundaries under internal/components/ ;
- there is a single maintainer and no divergent release cycles;
- validation must run across products (docs ↔ tooling ↔ skills), which is cheap in one repository and expensive across several.

## Decision

Adopt a **modular monolith inside a single monorepo**.

- One Go module ( github.com/SergioLacerda/rpg-system-rgb ) with one binary entrypoint per product surface under cmd/ as they materialize ( rgb , later rgb-compile , rgb-validate , rgb-query ).
- Each product is a component boundary under internal/components/<name> , isolated by enforced import rules (see below), so a future repository split is a directory move, not a re-write.
- Non-Go surfaces stay as monorepo siblings: docs/ (canonical content), skills/ (light-weight Markdown/YAML skills), web/ (Astro landing), generated/ (derived artifacts only), scripts/ (standalone validation module).

Boundaries are enforced mechanically, not by convention:

- `tests/architecture/` fails the build when a component imports `app`, `cmd`, or a sibling component;
- `.golangci.yml` depguard rules repeat the same constraints in the linter;
- `make review-structure` is the aggregate local gate ( `docs/engineering/base-structure-review-workflow.md` ).

## Split Criteria

Extract a product into its own repository only when at least one of these holds (mirrors the V2 analysis, section 20.3):

- release cycles genuinely diverge;
- distinct teams or communities own different products;
- independent distribution produces real benefit (e.g. a public SDK);
- repository size becomes an operational obstacle.

Because component boundaries are import-clean by construction, a split requires moving `internal/components/<name>` plus its `cmd/` surface and publishing the shared contract package — nothing else should break.

## Consequences

Positive:

- contracts between products stay synchronized in one place;
- cross-product validation (docs, schemas, bundles) runs in one gate;
- refactoring across boundaries is atomic;
- the architecture tests keep the monolith modular instead of letting it degrade into an entangled monolith.

Negative / accepted costs:

- one repository carries heterogeneous toolchains (Go, Node/Astro, Markdown pipelines);
- releases of independent products must be tagged within one history;
- contributors see the whole ecosystem even when touching one product.

# ADR-005: Zero External Dependency Posture

## Status

Accepted, with a documentation-PDF exception added on 2026-07-31.

## Context

RGB System V2 has never imported anything outside the Go standard library, in any of its Go modules, at any point in its history — no `go.sum` exists anywhere in the repository. This was never decided in a single place; it emerged as a consistent pattern across independent missions, each reasoning locally that a proposed dependency would be "the first" of its kind and rejecting it on that basis:

- `docs-compiler-html-pdf-scope-20260726` rejected automating PDF generation via headless-Chromium (an external binary requirement) or a Go-native PDF library (a new Go module dependency), keeping PDF export a documented manual browser step instead.
- `guardrails-tooling-scope-20260726` rejected adopting `gocognit` (cognitive-complexity linting) specifically because it requires installing a new external Go tool, even though the project already had build-failing cyclomatic-complexity coverage via `gocyclo` (bundled with the already-required `golangci-lint` dev tool, not a module dependency).

Each decision was locally reasoned and undocumented as a standing rule. This ADR formalizes the pattern those decisions were already following, so future missions can cite one authoritative source instead of re-deriving the same reasoning independently each time.

## Decision

**RGB System V2's Go modules do not take on external dependencies — no `go.sum` entries, no external binaries, no runtimes beyond the Go toolchain itself — unless a future ADR explicitly revisits this decision.**

This applies to:

- Go module dependencies (anything that would produce a `go.sum` entry);
- external binaries or runtimes invoked by any `make` target or `cmd/*` entrypoint (e.g. a headless browser, a Java runtime, a database server);

- build-time or render-time dependencies embedded into generated output (e.g. a JavaScript rendering library bundled into `generated/library/html/**`).

It does **not** apply to:

- developer tooling that is not a dependency of the module itself (`golangci-lint` is already required to run `make lint`, the same way a compiler is required to build; it does not appear in `go.sum` and does not ship inside any built artifact);
- plain-text, markup-embeddable formats with no rendering dependency of their own — see Consequences below for the Mermaid application of this principle.

## Consequences

- Every future proposal to add a Go module, external binary, or runtime dependency must be weighed against this ADR, not re-argued from first principles — and requires a new ADR to override, per this repo's own `M016`-equivalent guardrail-non-regression pattern (see `tests/architecture/architecture_test.go`'s own header comment and `docs/engineering/base-structure-review-workflow.md`). `docs-readme-c4-adr-scope-20260726` applied this reasoning directly: C4 architecture diagrams (`docs/architecture/**`) use Mermaid syntax embedded in plain Markdown — no PlantUML (requires a Java runtime or a rendering server) and no Go diagrams-as-code library (a new module dependency) — specifically because both alternatives would violate this ADR.
- This keeps the project's build and review surface small: `go build ./...` and `go test ./...` never need network access or a lockfile audit.
- The project accepts the corresponding cost: some features (automated PDF generation, cognitive-complexity linting, richer diagram styling) stay deferred or manual rather than automated, until a future ADR explicitly decides the trade-off is worth it for a specific case.

## Addendum 2026-07-31: Documentation PDF Generation Exception

Supersession note: this exception was retired for active publication by [ADR-010](#). The section below is kept as historical context for the temporary exception that existed before the Go-owned publication path replaced it.

The project accepts a narrow exception for automated PDF generation in the documentation publication pipeline.

This exception exists because the landing page and Library now expose stable PDF download links, and keeping those links fresh by manual browser export is operationally fragile. A generated PDF is a distribution artifact for readers, not a source of rules authority.

The exception allows:

- Python documentation-build dependencies required by a MkDocs PDF plugin;

- operating-system rendering libraries required by that plugin, when documented as setup pre-requisites;
- Makefile targets dedicated to documentation PDF generation, such as `docs-pdf` or `pdf-build`;
- generated PDF files under the landing static download surface.

The exception does not allow:

- Go module dependencies or any `go.sum` entry;
- runtime dependencies used by game/system code;
- headless-browser automation unless a later ADR explicitly authorizes that class of dependency;
- treating generated PDFs as canonical documentation sources;
- silently making every build depend on PDF rendering before the dependency profile is documented and validated.

Implementation constraints:

- Keep the PDF dependency profile separate from the base documentation profile, for example `requirements-docs-pdf.txt` in addition to `requirements-docs.txt`.
- Prefer an explicit `docs-pdf` or `pdf-build` target during stabilization. `landing-build` may depend on it only after the PDF dependency profile is reproducible in the supported development environment.
- Generated PDFs must be traceable to canonical Markdown or semantic source units through the existing projection/manifest model.
- The landing page should link to stable `latest` aliases while preserving versioned PDF files for release history.
- Release manifest generation, checksum verification, and PDF editorial validation are project-owned tooling and should run through Go. Python remains allowed here only for the MkDocs renderer stack until a later ADR retires or replaces that publication path.

# ADR-006: `cmd/rgb` As The Unified CLI

## Status

Accepted. Amended 2026-08-02 — see [Amendment: publication pipeline retirement](#) and [Amendment: deprecated caller migration](#) below.

## Context

A base project architecture review found the CLI surface ambiguous: `cmd/rgb` advertised scaffold readiness only (`app.Hello()`), while the functional commands were split across `cmd/rgb-tooling` (`validate`, `generate`, `bundle`) and `cmd/rgb-compiler` (`all`, `no-html`). Nothing distinguished which binary a user or CI job was supposed to run, and the review left the choice open: either promote `cmd/rgb` to a unified CLI, or keep it as a status command and document the other two as the real CLIs.

## Decision

`cmd/rgb` is the unified CLI. It now dispatches to the same `internal/app` use cases the other two binaries already called:

```
rgb                    -> scaffold/component status (unchanged)
rgb validate [repo-root] -> internal/app.ValidateDocs
rgb generate [repo-root] -> internal/app.GenerateProjections
rgb bundle  [repo-root]  -> internal/app.BuildBundle
rgb compile  all|no-html [repo-root] -> internal/app.Compile
```

`Makefile` targets (`validate`, `generate`, `bundle`, `compile`, `compile-full`) now invoke `cmd/rgb` instead of the two split binaries.

`cmd/rgb-tooling` and `cmd/rgb-compiler` are **deprecated, not removed**. They still build and behave exactly as before because real callers depended on them at the time of this decision: `tests/semantic_docs/semantic_docs_test.go` shelled out to `go run ./cmd/rgb-tooling validate`, and `docs/core/semantic/README.md` documented `rgb-tooling` directly. Removing a working entrypoint those depended on was a breaking change under M006 (RFC Process for Breaking Changes) and was out of scope for the original decision.

## Consequences

Positive:

- one documented entrypoint for the product-facing CLI surface, matching the "one coherent CLI" gap called out by the architecture critique;
- `internal/app` stays the single seam between entrypoints and components, so `cmd/rgb`'s dispatch logic is a thin, independently testable layer (`cmd/rgb/main_test.go`) with no new component coupling.

Negative / accepted costs:

- two deprecated binaries remain in the tree until their callers migrate;
- `cmd/rgb`'s subcommand set must be kept in sync with `internal/app` by hand until there's enough surface to justify a flag-parsing library (see ADR-005 on the zero external dependency posture for why that's not a default reach).

## Follow-up

- Completed: `tests/semantic_docs/semantic_docs_test.go` and `docs/core/semantic/README.md` now call `cmd/rgb`.
- Remaining: propose `cmd/rgb-tooling` removal through the M006 RFC process if and when compatibility support is no longer needed. (`cmd/rgb-compiler` no longer applies here — see the publication-pipeline amendment.)

## Amendment: Publication Pipeline Retirement (2026-08-02)

A base project Library review found that, unlike `cmd/rgb-tooling`, `cmd/rgb-compiler` and the `compile` subcommand had **no real caller** — no test, CI workflow, or required Makefile target used them; `internal/components/compiler` and `internal/components/library` underneath them were equally uncalled outside that path. M006's breaking-change concern (real callers depend on this) did not apply, so [ADR-008](#) removed `compile`, `cmd/rgb-compiler`, `internal/components/compiler`, and `internal/components/library` outright instead of deprecating them. Every mention of `compile` / `cmd/rgb-compiler` above describes this ADR's original decision at the time it was written, not the current subcommand set — see ADR-008 for what replaced it. The `cmd/rgb-tooling` decision above is unaffected.

## Amendment: Deprecated Caller Migration (2026-08-02)

The repository callers named in the original decision have migrated to `cmd/rgb`: `tests/semantic_docs/semantic_docs_test.go` runs `go run ./cmd/rgb validate`, and `docs/core/semantic/README.md` documents `cmd/rgb` as the equivalent command for `make validate` and `make generate`.

`cmd/rgb-tooling` remains present only as a compatibility path. Removing it is still a breaking-change decision and requires the M006 RFC process.

# ADR-007: Bundle Expansion Requires A Consumer Contract First

## Status

Accepted.

## Context

`internal/components/bundles` currently projects a minimal, real subset of the semantic index (`id`, `kind`, `title`, `source_status`, `relationships`) into `generated/bundle/rgb.bundle.json`. A base project architecture review confirms this is a useful minimum but flags that the plan's full bundle shape (`rules`, `procedures`, `graph`, `search`, locale packs, validation reports) has no defined consumer yet:

- `internal/components/maker` and `internal/components/specialist` are boundary stubs only (`Descriptor()` and nothing else) — there is no running code to derive a real field-level contract from;
- MkDocs + Astro (the actual Library/PDF publication path) consumes canonical `docs/` Markdown and the semantic index directly today, not the bundle — `internal/components/library` was retired by [ADR-008](#) for being an uncalled parallel path, not a bundle consumer to design a contract for;
- no external/third-party consumer exists yet.

Both the review's proposal and design notes are explicit: expanding the bundle before consumers are defined risks producing "attractive but unused generated artifacts," and non-goals explicitly forbid treating the Maker/Specialist stubs as if they were implemented products.

## Decision

Bundle expansion is **contract-first**: no field is added to `Bundle` / `BundleUnit` (`internal/components/bundles/bundles.go`) for a given consumer until that consumer has a written contract stating:

1. which bundle fields it reads and why (not "might use");
2. the read pattern (one-shot load, incremental query, streaming);
3. a compatibility expectation (can it tolerate additive-only changes, or does it need a schema version bump on any change);
4. a test that exercises the consumer against a bundle fixture, per `tasks.md`'s "add tests for any new CLI or bundle contract before implementation."

This repo does not yet have a contract meeting that bar for any consumer. Concretely, that means:

- Maker and Specialist stay boundary stubs until their first real use case exists; writing a bundle contract for a stub product would be speculation, not a contract.
- Library's future bundle consumption (if any) needs its own contract before `bundles` grows fields for it — Library reading `docs/` directly today is not itself a reason to add unrelated fields.
- External/third-party consumers need the same bar; `bundleSchema = "rgb-system-bundle/0.1"` stays additive-compatible (new optional fields only) until a real external consumer forces a versioning decision.

## Consequences

Positive:

- prevents the bundle from accumulating unused speculative structure;
- keeps `bundles` importing only `internal/components` (the shared leaf contract), per the existing `TestComponentsDoNotImportSiblings` boundary — a contract-first bundle has no reason to reach into a sibling component to guess at its needs.

Negative / accepted costs:

- Maker/Specialist integration stays blocked until product work defines what they actually need, which this ADR does not do — it is a process gate, not a schedule.

## Update (2026-08-27)

This ADR's Context section evaluated `internal/components/bundles` (the Go component) but missed a pre-existing, parallel artifact: `docs/core/semantic/projection-manifest.v0.1.json` already declared a `projection.bundle.core-rules.v0_1` projection (added 2026-07-26, a week before this ADR), claiming `"owner": "bundles"` and producing `generated/bundles/core-v2-rules.bundle.json` with a shape already larger than the minimal one described above — the same "attractive but unused generated artifact" risk this ADR

warns against, just via a different code path. It had no real consumer (confirmed: no non-test Go code read it) and was never promoted past `"status": "proposed"`. The entry has been removed from the projection manifest; this Decision's contract-first policy now applies without exception across both the `bundles` component and any future `owner: bundles` projection-manifest entries.

## Non-negotiable: component import boundaries

Independent of bundle scope, `docs/architecture/components.md`'s enforced rule set (`tests/architecture/architecture_test.go`: `TestComponentsDoNotImportOuterLayers`, `TestComponentsDoNotImportSiblings`, `TestSharedContractStaysLeaf`, `TestEntrypointsGoThroughApp`) stays exactly as-is. No bundle contract, CLI change, or integration work may introduce a sibling-to-sibling import (e.g. `maker` importing `bundles` directly) to shortcut this policy — it must go through `internal/app`, unchanged from ADR-004's original boundary decision.

# ADR-008: Retire The Go HTML/Print/PDF Publication Pipeline

## Status

Accepted.

## Context

The base project Library review found that RGB Library publication actually happens through MkDocs (Library HTML) → Astro (static hosting) → generated PDF downloads, and flagged that `internal/components/library` still described an older, separate publication concept that could mislead future work into treating it as the active path.

Investigating the fate of `internal/components/library` surfaced a wider finding: `internal/components/compiler` (a hand-written ~1,150-line Markdown parser and HTML/print renderer, with its own tests) and the `cmd/rgb-compiler` binary formed the same orphaned pipeline. A repository-wide search found zero consumers of either package's output — no CI workflow, Makefile target required by `check / review-structure / check-fast`, MkDocs config, or Astro/TypeScript file referenced `generated/library/html/**`, `generated/library/print/**`, `generated/library/PDF_EXPORT.md`, or any `compiler / library` package function. `make compile` and `make compile-full` were documented in `README.md` but not wired into any gate. `cmd/rgb-compiler` (unlike `cmd/rgb-tooling`, see [ADR-006](#)) had no real caller either — no test shelled out to it.

This is a different situation from ADR-006's `cmd/rgb-tooling / cmd/rgb-compiler` deprecation: there, real callers existed (`tests/semantic_docs/semantic_docs_test.go`, `docs/core/semantic/README.md`) so the binaries were kept working and only marked deprecated. Here, no real caller existed for the compile path at all — keeping it working would have preserved dead weight, not backward compatibility.

## Decision

Retire the Go HTML/print/PDF pipeline outright, not deprecate it:

- Delete `internal/components/library` (`library.go`, `site.go`, `pdf_export.go`, and their tests).
- Delete `internal/components/compiler` (Markdown parser, `block.go`, `render_html.go`, `render_print.go`, `render_tree.go`, and their tests).
- Delete `internal/app/compile.go` (`app.Compile`).
- Remove the `compile` subcommand from `cmd/rgb` (`cmd/rgb/main.go`); `rgb compile ...` now returns `unknown subcommand "compile"`.
- Delete `cmd/rgb-compiler` entirely (no ADR-006-style deprecation — it had no real caller to protect).
- Remove `compile` / `compile-full` from `Makefile` and their `README.md` documentation.
- Drop `library` / `compiler` from `internal/app.Components()` and the architecture diagrams (`docs/architecture/components.md`, `docs/architecture/containers.md`).

The documented publication path at the time of this decision was the interpreter-backed Library/PDF path plus Astro. ADR-010 later replaced that active path with `cmd/rgb docs ...` plus Astro, without reviving the deleted packages or the removed `rgb compile` command.

## Consequences

Positive:

- removes a substantial parallel implementation (custom Markdown parser + two renderers + site/PDF-instruction writer) that had zero real consumers, eliminating the exact drift risk the Library review warned about ("two publication narratives that should be reconciled");
- `internal/app.Components()` and the architecture diagrams now describe only components with an active call path from `cmd/*`.

Negative / accepted costs:

- this is a real deletion of working, tested code (not a stub) — reversible only through git history, not through a runtime flag or config toggle;
- if a future need for programmatic (non-MkDocs) HTML/print rendering emerges, it will be rebuilt rather than resumed from this code.

## Relationship To ADR-006

ADR-006 originally described `cmd/rgb`'s `compile` subcommand and `cmd/rgb-compiler`'s deprecation-not-removal alongside `rgb-tooling`. This ADR supersedes that part of ADR-006: `compile` and `cmd/rgb-compiler` are removed, not deprecated. ADR-006's decision about `cmd/rgb-tooling` (kept deprecated, real callers) is unaffected and still stands.

# ADR-009: Retain The MkDocs/Python Publication Exception Until Replacement Parity

## Status

Superseded by [ADR-010](#).

Historical status before supersession: Accepted.

## Context

ADR-005 established the project's zero external dependency posture and then added a narrow documentation-PDF exception on 2026-07-31. That exception allows Python documentation-build dependencies required by the MkDocs PDF plugin, operating-system rendering libraries documented for that plugin, and generated PDFs under the landing download surface.

The project-owned release validation path has since been moved to Go: release manifest generation, checksum verification, PDF metadata checks, TOC checks, and raster smoke checks no longer depend on Python or Pillow. That leaves the renderer/publication stack itself as the remaining Python surface:

- `make docs-build` uses `python3 -m mkdocs` for the HTML Library;
- `make docs-pdf` uses MkDocs plus `mkdocs-to-pdf` for bilingual PDF downloads;
- CI uses `actions/setup-python` for documentation and deployment jobs;
- `docs-build/requirements-docs.txt` and `docs-build/requirements-docs-pdf.txt` define the Python dependency profiles;
- Dependabot tracks the `/docs-build` `pip` ecosystem while the exception remains active.

ADR-008 is also binding context. It retired the previous Go HTML/print/PDF pipeline because it had no active caller and would have preserved a parallel, orphaned publication narrative. A future non-Python renderer must therefore be a new justified architecture decision with real consumers and parity evidence, not a silent resurrection of `internal/components/compiler`, `internal/components/library`, `cmd/rgb-compiler`, or `rgb compile`.

## Decision

Keep the MkDocs/Python publication exception for now.

The current publication topology remains:

```
docs/**
|
|-- MkDocs strict build -----> web/landing/public/library/**
|
+-- MkDocs + mkdocs-to-pdf + Poppler ---> web/landing/public/downloads/*.pdf
                                         |
                                         v
                                   Go-owned release artifact gates
```

Retiring Python from documentation publication requires a later ADR or an explicit amendment to this ADR. That later decision must identify the replacement topology and provide parity evidence before deleting the current Python surfaces.

The replacement must preserve or deliberately supersede all current public release behavior:

- HTML Library generation;
- Astro landing packaging of the Library;
- English and PT-br PDF generation;
- stable `latest` PDF aliases;
- versioned PDF files;
- release manifest generation;
- aggregate `SHA256SUMS` and per-version `.sha256` files;
- PDF editorial smoke checks.

Until that parity exists, do not remove or weaken:

- `PYTHON`, `MKDOCS`, `docs-install`, `docs-pdf-install`, `docs-build`, or `docs-pdf` in the Makefile;
- `docs-build/requirements-docs.txt`;
- `docs-build/requirements-docs-pdf.txt`;
- Python setup and pip caching in CI documentation/deployment jobs;
- the Dependabot `pip` entry for `/docs-build`.

## Consequences

Positive:

- keeps the current public documentation and PDF release path stable;
- avoids deleting a working renderer without a proven replacement;
- keeps ADR-005's exception narrow and explicit;
- prevents accidental reversal of ADR-008 by forbidding a silent revival of the deleted Go compiler/library pipeline;

- leaves release artifact validation Go-owned and independent from the Python renderer exception.

Negative / accepted costs:

- the repository still needs Python for documentation publication;
- `docs-build/` remains a dependency surface that CI and Dependabot must track;
- fully zero-Python publication remains deferred until a later ADR selects and proves a replacement topology.

## Replacement Requirements

Any future proposal to retire MkDocs/Python must include at least:

1. A renderer topology that does not reintroduce the ADR-008 deleted pipeline by implication.
2. Local validation equivalent to or stronger than:
3. `make docs-build`;
4. `make landing-build`;
5. `make docs-pdf`;
6. `make release-artifact-manifest`;
7. `make release-artifact-check`;
8. `make pdf-editorial-check`.
9. CI changes proving the replacement runs without `actions/setup-python`, pip cache, or `docs-build/requirements-*.txt`.
10. A deletion plan for obsolete Python publication files only after parity passes.
11. Updated public documentation describing the new publication path.

Full PDF/UA tagging, durable visual-regression baselines, and deeper accessibility checks remain future quality work. They are desirable, but they are not prerequisites for keeping the current MkDocs/Python exception active.

# ADR-010: Go-Owned Publication Without Python

## Status

Accepted.

## Context

ADR-005 established the project's zero external dependency posture and later allowed a narrow documentation publication exception for the former renderer stack. ADR-008 removed the orphaned Go HTML/print/PDF compiler path. ADR-009 then kept the former exception only until a replacement topology proved parity.

The remaining interpreter-backed surface was publication:

- documentation HTML rendering;
- PDF download publication;
- CI setup for the renderer;
- dependency update tracking for that renderer.

Release metadata and editorial validation were already Go-owned through `cmd/rgb release manifest`, `cmd/rgb release check`, and the tooling package.

## Decision

Retire the interpreter-backed publication exception and make publication Go-owned.

The active topology is now:

```
docs/core/**
|
+-- cmd/rgb docs library -----> web/landing/public/library/**
|
+-- cmd/rgb docs pdf -----> web/landing/public/downloads/**
                             |
                             v
                        cmd/rgb release manifest/check
```

Astro remains the static landing package and hosting surface. It packages the generated public Library and PDF downloads from `web/landing/public/**`.

This is a new publication boundary, not a revival of the ADR-008 deleted compiler/library packages or the removed `rgb compile` command.

## Consequences

Positive:

- project publication no longer installs or runs the retired interpreter stack;
- CI and local Make targets use Go plus the existing Astro/Node landing workspace;
- dependency update tracking no longer needs the retired renderer ecosystem;
- the public Library is generated through the unified `cmd/rgb` CLI topology;
- release manifest and checksum behavior remains Go-owned.

Accepted costs:

- the Go HTML renderer intentionally supports a constrained Markdown subset based on the current public `docs/core/**` files;
- PDF publication is a Go-owned publishing step for reviewed PDF assets, not a full general-purpose PDF layout engine;
- Poppler remains an external command-line validation dependency for PDF editorial smoke checks. It is not part of publication rendering and does not reintroduce the retired interpreter runtime.

## Replacement Parity

The replacement preserves:

- HTML Library generation from canonical `docs/core/**`;
- Astro packaging of the Library;
- English and PT-br public documentation surfaces;
- stable `latest` PDF aliases;
- versioned PDF files;
- release manifest generation;
- aggregate `SHA256SUMS` and per-version `.sha256` files;
- PDF editorial smoke checks.

## Supersession

This ADR supersedes ADR-009 for active publication behavior. ADR-009 remains as the historical record of why the exception was retained until this replacement existed.

# ADR-011: Split License Model

## Status

Accepted.

## Context

The repository contains two materially different kinds of work:

- executable software surfaces such as Go packages, shell scripts, CI, and Astro application code;
- creative and rules-authority surfaces such as RPG rules, examples, documentation, diagrams, public copy, generated Library/PDF content, and art.

The prior single CC BY 4.0 repository license was suitable for creative rules text, but it was less conventional for software reuse and downstream tooling.

## Decision

Adopt a split model:

| Surface                                                                             | License           |
|-------------------------------------------------------------------------------------|-------------------|
| Source code, scripts, CI, and tooling                                               | MIT or Apache-2.0 |
| Rules text, examples, documentation, diagrams, web copy, generated content, and art | CC BY 4.0         |

`LICENSES.md` is the repository surface map. The root `LICENSE` file points to the concrete license texts.

## Consequences

- Software consumers can use the code under a conventional permissive software license.
- Rules and documentation retain attribution requirements appropriate for creative RPG content.
- New files should be classified by surface ownership before publication.
- Ambiguous generated artifacts inherit the license of their source surface.

# ADR-012: MIT-Only License

## Status

Accepted.

## Context

ADR-011 adopted a split licensing model: source code, scripts, CI, and tooling were dual-licensed under MIT or Apache-2.0, at the user's option, while rules text, examples, documentation, diagrams, web copy, and generated Library/PDF content were licensed under Creative Commons Attribution 4.0 International (CC BY 4.0), which requires attribution.

The goal has since narrowed: a single, maximally permissive license across the entire repository, with unrestricted open-source use, and with the copyright notice itself naming the actual author rather than only the project name.

## Decision

Adopt MIT as the sole license for the entire repository — code, rules text, documentation, diagrams, web copy, generated content, and art alike. The Apache-2.0 option and the CC BY 4.0 license are retired from active use.

The root `LICENSE` file holds the full MIT license text directly (not an index to separate license files), with:

```
Copyright (c) 2026 Sergio Lacerda (RGB System)
```

`LICENSE-MIT`, `LICENSE-APACHE-2.0`, `LICENSE-CC-BY-4.0`, and `LICENSES.md` are removed — there is no split model left to map or duplicate.

## Consequences

- The repository's licensing surface collapses to a single `LICENSE` file.

- Anyone may use, copy, modify, merge, publish, distribute, sublicense, and sell all repository content, subject only to MIT's copyright-notice retention condition — matching the request for unrestricted open-source access.
- The CC BY 4.0 attribution *requirement* for rules text, examples, docs, diagrams, and art is dropped. This is an explicit, accepted trade-off in favor of being fully unrestricted, not an oversight.
- The Apache-2.0 explicit patent grant is no longer offered for code.
- All prose references to the split model ( `README.md` , `docs/README-details.md` ) are updated to point at `LICENSE` directly.

## Supersession

This ADR supersedes ADR-011. ADR-011 remains as the historical record of why the split model was originally adopted.

# ADR-013: Skill Distribution Via The Existing Publication Pipeline

## Status

Accepted.

## Context

A proposal to distribute the RGB Specialist skill as a downloadable `.zip` (captured in an internal pending-analysis note) suggested a git-tag-triggered GitHub Actions job publishing the archive as a GitHub Release asset, with a dedicated install page and shell commands to unzip it into an `~/.agents/skills`-style location.

This repository has no git-tag-triggered release job and no `action-gh-release`-style step anywhere in `.github/workflows/ci.yml`. Publication instead happens on every push to `main`: a `deploy` job bundles generated documentation artifacts into a single static site (`web/landing/dist`) and publishes it to GitHub Pages via `actions/upload-pages-artifact` + `actions/deploy-pages`.

ADR-010 already established that topology for the two existing publication outputs:

```
docs/core/**
|
+-- cmd/rgb docs library -----> web/landing/public/library/**
|
+-- cmd/rgb docs pdf -----> web/landing/public/downloads/**
                                |
                                v
                                cmd/rgb release manifest/check
```

ADR-008 separately established that this project retires orphaned, parallel publication pipelines rather than let them accumulate alongside the active one. Adding a GitHub-Release-based flow for skill distribution would reintroduce exactly that pattern: a second publication mechanism, with its own trigger, permissions, and versioning scheme, running alongside the Pages-based one that already serves the Library and PDFs.

Separately, `cmd/rgb` is this project's unified CLI (ADR-006), and it is never invoked by end users directly — only by `make` targets and CI jobs. Any skill distribution mechanism needs to preserve that: no new user-facing installer CLI for this project.

`internal/components/tooling/release_artifacts.go`'s `WriteReleaseArtifactManifest / CheckReleaseArtifacts` — the existing manifest/checksum machinery for PDF downloads — hardcodes an expected file list of exactly four PDF files (two locales, `latest` + versioned) and runs PDF-specific editorial validation (`pdfinfo` / `pdftotext` / `pdftohtml` / `pdftoppm`, page-count, TOC-link, and raster-luminance checks). It is not a generic "checksum any artifact set" utility and cannot be pointed at a skill `.zip` without change.

## Decision

Skill distribution joins the existing Go-owned publication topology (ADR-010) as a third artifact type, published the same way the Library and PDFs already are — not via GitHub Releases:

```
skills/<skill>/**
|
+-- cmd/rgb docs skill -----> web/landing/public/downloads/**
                                |
                                v
                        a lightweight skill-artifact manifest/checksum step
                        (separate from, not a reuse of, the PDF-specific
                        release manifest/check machinery above)
```

- A skill package (e.g. `skills/specialist/`) is zipped by a new `cmd/rgb docs skill` subcommand, with the archive's top-level entry being the skill's own directory name — never a repo-root-relative or auto-generated wrapper folder.
- The `.zip` (`<name>-<version>.zip`, `<name>-latest.zip`) is written to `web/landing/public/downloads/`, the same directory the PDFs already use, and is published as a static file through the existing push-to-`main` → GitHub Pages `deploy` job. No new CI trigger, secret, or permission is introduced.
- Manifest/checksum generation for the skill `.zip` is a new, minimal step (SHA256 + byte count only), deliberately kept separate from `WriteReleaseArtifactManifest / CheckReleaseArtifacts` rather than generalizing those functions — the PDF checker's entire design center is PDF-specific editorial validation that does not apply to a skill archive, and coupling the two risks regressing PDF checks when the skill path changes.
- No new CLI is introduced for end users. `cmd/rgb docs skill` is invoked the same way `cmd/rgb docs library` / `cmd/rgb docs pdf` already are: through a Makefile target, from CI, never directly by a person installing the skill.

## Consequences

### Positive:

- skill distribution reuses an already-reviewed publication boundary instead of introducing a second one;
- no new GitHub Actions triggers, permissions, or secrets;
- naming/versioning for the skill archive mirrors the existing PDF convention ( `<name>-<version>`, `<name>-latest` ), keeping the public `downloads/` directory self-consistent;
- the PDF-specific release-manifest/check machinery stays focused on PDF editorial validation and is not stretched to cover an unrelated artifact shape.

### Accepted costs:

- a new, separate manifest/checksum code path is needed for the skill `.zip` rather than a single shared implementation across all downloadable artifacts;
- distribution here covers packaging/publication only — skill package *content* (what `skills/specialist/` actually contains, and whether it has a runnable contract) is defined by separate, independent work and is out of scope for this decision;
- multi-runtime skill packaging (for example a Codex- or Gemini-specific layout) is explicitly out of scope; this decision targets the Claude Code skill-loading convention already used by `skills/specialist/` and `skills/maker/` in this repository.

## Relationship To Prior ADRs

This ADR extends ADR-010's topology rather than amending it: ADR-010's Library/PDF flow is unchanged, and this decision adds a third, independent branch alongside it. It reaffirms ADR-008's rejection of parallel/orphaned publication pipelines and ADR-006's unified-CLI boundary ( `cmd/rgb` is never an end-user-facing installer).

# ADR-014: RGB Specialist Bundled References Snapshot

## Status

Accepted.

## Context

`skills/specialist/` grounds its answers in `generated/ai-context/core-specialist-pack.json` and `docs/core/semantic/**` (see ADR-002). Both are repo-relative: if the `skills/specialist/` folder is copied outside this monorepo — which ADR-013 now makes possible by publishing it as a downloadable `.zip` — it loses access to the canonical rule documentation it depends on to answer questions and cite a source.

The user asked for the package to be self-contained, carrying canonical documentation material with it, rather than depending on sibling paths in this monorepo.

## Decision

`skills/specialist/references/rgb-system.md` is added as a bundled copy of the four canonical rule documents (`docs/core/en/**` and `docs/core/PT-br/**`), consolidated verbatim with each section labeled by its original source path.

This is a **hand-maintained snapshot**, not a generated artifact:

- `docs/core/**` remains the single source of truth (ADR-002) — the bundled copy never overrides it;
- `SKILL.md`'s grounding-source precedence uses the bundled copy only as a fallback (inside the monorepo) or as the sole available source (running standalone, outside the monorepo) — citation is still required in every case;
- no automated regeneration step exists yet for this file (unlike `generated/ai-context/*`, which is regenerated via `make generate`). Adding one is a config/tooling change, deferred

to a separate task (Task 6 of the parallel skill-materialization mission) — it touches `con-fig.yaml`, which is out of this mission's documentation-only scope.

## Consequences

Positive:

- the skill package answers questions and cites a source even when copied outside this monorepo, satisfying the self-contained requirement without changing the existing consumer contract or procedures;
- the precedence order keeps the higher-trust, ID-traceable sources (compiled pack, semantic index) as first choice whenever they are available, so nothing regresses for in-monorepo use.

Accepted costs:

- `references/rgb-system.md` and `docs/core/**` can drift until the automation from Task 6 exists — anyone updating `docs/core/**` should know the bundled copy needs a manual refresh until then;
- this is intentionally scoped to content only. It does not decide how the folder gets packaged or published; that is ADR-013's decision.

## Update (2026-08-27)

Task 6 is closed: `skills/specialist/references/rgb-system.md` is now regenerated by `make generate` (`internal/components/tooling.GenerateSkillSnapshot`, reading the input list from `skills/specialist/config.yaml`'s `sources.bundled_snapshot_inputs`) and drift-checked in CI (`scripts/ci/check-generated-drift.sh`), which `deploy` already depends on. The "accepted cost" above — drift between this file and `docs/core/**` until Task 6 landed — no longer applies: the file is a generated artifact now, not hand-maintained, and a stale snapshot blocks deployment instead of publishing silently.

## Relationship To ADR-013

ADR-013 decided how `skills/specialist/` gets zipped and published (reusing the existing Pages-based publication pipeline). This ADR decides what the folder needs to contain to still be useful once distributed that way — a self-contained copy of the canonical rules it grounds answers in. The two are independent and do not amend each other: ADR-013 governs packaging/publication, this ADR governs package content.

# ADR-015: Skill Runtime Split — LLM Instruction Layer vs. Go Component Layer

## Status

Accepted.

## Context

`skills/specialist/**` and `skills/maker/**` both ship a complete skill contract (`SKILL.md`, procedures/schemas, templates, config) but neither has a working runtime. `internal/components/specialist/specialist.go` and `internal/components/maker/maker.go` are both 13-line stubs exposing only a `Descriptor()` — there is no behavior on either side of the boundary today.

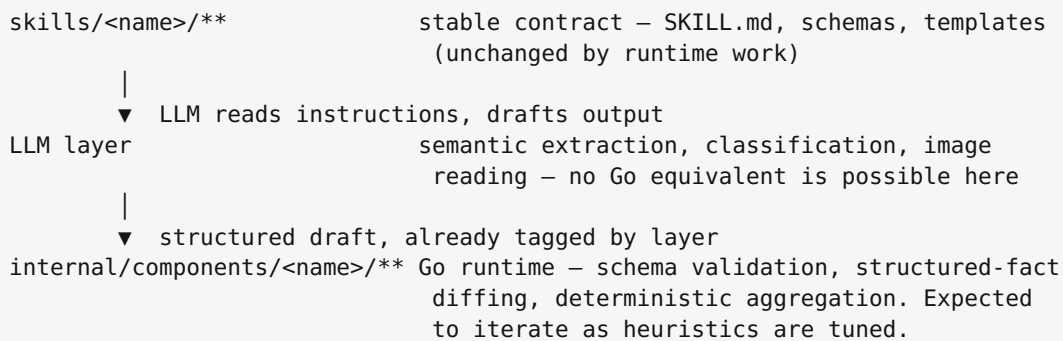
The accepted Specialist-first skill roadmap gated Maker's runtime work behind three conditions: (a) Specialist has a source-trace benchmark, (b) the bundle/search context shape is stable, (c) Maker's provenance/canon schemas are accepted. A 2026-08-27 runtime-base review re-checked these against the current repository:

- (b) and (c) are met — `generated/ai-context/core-specialist-pack.json` / `docs/core/semantic/core-v2.index.json` are in stable use, and Maker's four schemas (`entity-package`, `maker-report`, `visual-observation`, `conflict-report`) exist and are internally consistent.
- (a) is met only **structurally** — `skills/specialist/benchmark/golden-qa.yaml` plus `tests/fixtures/specialist_golden_qa_test.go` validate that the benchmark dataset cites real semantic IDs, but no Specialist runtime exists to exercise the benchmark behaviorally, because Specialist has no runtime either.

That review also found that "use Specialist as the base" only holds for the contract layer (`SKILL.md` / schemas/procedures shape) — there is no proven runtime pattern in this repo to copy, for either skill.

## Decision

Split any skill's runtime, once it needs one, into two layers:



Applied to Maker specifically, per package kind:

| Kind               | LLM layer                           | Go layer ( <code>internal/components/maker/**</code> )                |
|--------------------|-------------------------------------|-----------------------------------------------------------------------|
| Entity Package     | extract facts/inferences from notes | validate against <code>entity-package.schema.yaml</code>              |
| Maker Report       | thin prose rollup                   | aggregate counts over already-classified packages                     |
| Visual Observation | read appearance from an image       | enforce appearance-only field scope (image never claims a fact field) |
| Conflict Report    | flag disagreement during extraction | diff structured fields across packages sharing an entity ID           |

The contract layer ( `skills/maker/**` ) is the stable surface and is not expected to change as part of this work. The Go component layer ( `internal/components/maker/**` ) is where iteration is expected — differ heuristics, aggregation buckets, and validation strictness will be tuned as real usage surfaces edge cases.

Gate status: conditions (b) and (c) of the Maker Deferral Criteria are treated as met; condition (a) is treated as met for starting Maker's Go-side runtime work (validators, differ, aggregator — none of which require a working Specialist runtime to build or test), but not as evidence that Maker's own LLM-side behavior has been benchmarked.

## Consequences

Positive:

- Isolates expected instability (differ/aggregator/validator heuristics) to one package boundary, so iterating on it does not put the already-accepted `skills/maker/**` contract at risk of drifting out of sync with runtime behavior.

- Establishes a pattern reusable by Specialist (and any future skill) once it grows a real runtime, rather than a Maker-specific one-off.
- Continues the split already used for Specialist's existing tooling (bundle generation and the golden-qa structural check are Go; explaining a rule is LLM) instead of introducing an unrelated third pattern.

Negative / accepted costs:

- No implementation of this split exists yet for either skill — this ADR sets policy ahead of a first concrete build.
- Building `internal/components/maker/**`'s validators, differ, and aggregator, and wiring Maker's LLM-side procedures into an invocable runtime, remain open implementation work — out of scope for this ADR and for the Strategist mission that produced it (implementation\_handoff, not documentation).

## Non-Goals

This ADR does not authorize or perform any Go source change under `internal/components/maker/**`, nor any change to `skills/maker/**`'s existing contract. It records the architectural decision only.

# ADR-016: Reinstate A Narrow External PDF Authoring Exception

## Status

Accepted.

## Context

A 2026-08-27 PDF editorial-quality review re-examined a pending critique that proposed adding a headless-Chromium/Playwright HTML-to-PDF pipeline to achieve "editorial quality" (cover, table of contents, RGB-vector rule callout boxes) for the RGB Library PDF.

Two things emerged from direct verification, not just from reading the critique:

1. **Most of the claimed gap does not exist.** Inspecting the currently shipped PDF ( `web/landing/public/downloads/rgb-system-core-v2-*.pdf` ) directly — via `pdftotext` , `pdftohtml -xml` , and a rasterized page render — shows it already has a correctly paginated table of contents, 146 working internal hyperlinks on pages 1–3 alone, and running headers/footers ( `page/total` ). Only two things are actually missing: a designed cover (no RGB vector glyph, no version/license/date metadata — plain text only) and colored R/G/B rule callout boxes.
2. **The proposed fix conflicted with existing decisions.** The headless-Chromium pipeline directly conflicts with [ADR-005](#) (which already named and rejected headless-Chromium PDF generation once, in mission `docs-compiler-html-pdf-scope-20260726` ) and [ADR-010](#) (which retired the interpreter-backed publication path and states PDF publication is "a Go-owned publishing step for reviewed PDF assets, not a full general-purpose PDF layout engine").

Deeper verification found the real problem underneath: the shipped PDF's own metadata reports `Producer: WeasyPrint 62.3` , and `git log` traces it to a pre-ADR-010 commit ( `c2d2a5e` , "implement automated PDF generation workflow with MkDocs"). A repository-wide search finds zero remaining trace of that toolchain — no `mkdocs.yml` , no `requirements-docs*` , no `weasyprint` or `mkdocs-pdf` reference anywhere. The tool that produced today's good-quality PDF body was fully deleted when ADR-010 retired the automated publication path, without a documented re-

placement for *authoring* PDF content (as opposed to *publishing* it, which ADR-010's Go-owned pipeline still does correctly). Nobody today has a reproducible way to produce the next PDF release at all, cover/callouts aside.

ADR-005's original 2026-07-31 addendum already permitted exactly this class of exception (Python documentation-build dependencies, OS rendering libraries, a dedicated `docs-pdf / pdf-build` Make target, generated PDFs under the landing download surface) before ADR-010 retired it on the premise that the Go-owned replacement had reached parity. That parity claim holds for copy/validate/manifest/checksums, but not for authoring the PDF body's editorial content — nothing replaced MkDocs+WeasyPrint's role there.

## Decision

Reinstate a **narrow, external, non-Go, non-CI-blocking** PDF authoring exception:

- A PDF authoring tool of this class (MkDocs + WeasyPrint, or an equivalent the maintainer chooses) may run **outside** the Go module graph and outside the default `make /CI` build, as a dedicated, manually-invoked release step — producing one reviewed PDF per release, per language.
- That authoring step's template must add what's actually missing: a designed cover (RGB vector glyph, version, language, license, release date) and CSS-based callout/admonition styling for R/G/B rule boxes. It does not need to reproduce anything else — the existing TOC pagination, internal links, and running headers/footers already work and must be preserved, not rebuilt.
- The resulting reviewed PDF is supplied to the existing, unchanged Go pipeline via `cmd/rgb docs pdf --source-en / --source-pt-br (make docs-pdf)`, exactly as today. `internal/components/publication/pdf.go`'s copy-and-validate behavior does not change.
- `internal/components/tooling/release_artifacts.go`'s Poppler-based editorial smoke checks (`pdftotext / pdftohtml / pdftoppm`) remain the Go-owned validation gate over whatever the external tool produces.

This does **not** allow:

- any new Go module dependency (`go.sum` entry) for PDF generation or manipulation;
- headless-browser automation as part of the Go/CI build;
- treating the external authoring tool's output as canonical documentation — canonical source remains `docs/core/**` Markdown, unchanged (ADR-001);
- making any default build target (`make build`, `make test`, `make release-check`'s non-PDF gates) depend on the external tool's presence.

## Consequences

Positive:

- Closes a real reproducibility gap: the next PDF release has a documented path again, instead of depending on tooling that was silently deleted.
- Scopes the actual work correctly: template/cover/callout design, not a new pipeline or a new class of Go/CI dependency.
- Keeps ADR-010's substance intact — Go still owns copy, validate, manifest, and checksums; only PDF *authoring* (never publication) sits outside Go.
- Prevents re-litigating the headless-Chromium-vs-ADR-005 conflict from scratch the next time PDF editorial quality comes up.

Negative / accepted costs:

- Reintroduces an external, non-Go toolchain dependency for PDF authoring — the same tradeoff ADR-005's original addendum already accepted, now scoped more narrowly (cover + callout templates only, not full pipeline automation).
- Setting up the authoring tool, and designing the cover/callout templates, remain open implementation/authoring work — out of scope for this ADR and for the Strategist mission that produced it.

## Non-Goals

This ADR does not authorize any Go module dependency, any change to

`internal/components/publication/**` or `internal/components/tooling/**`, or any change to the canonical Markdown authority model (ADR-001). It records the authoring-exception decision only; the toolchain setup and the actual next PDF release are separate, unauthorized-by-this-ADR implementation work.

## Supersession

This ADR narrows and partially reinstates the addendum ADR-010 retired — scoped specifically to PDF *authoring* (cover/callout templates), not to the full automated interpreter-backed publication pipeline ADR-010 removed. ADR-010's Go-owned publication decision remains otherwise unchanged.

# Architecture Decision Records

This directory contains project-wide architecture decisions for RGB System.

- [ADR-001: AI-First Documentation Authority](#)
- [ADR-002: RGB Core V2 Design Control](#)
- [ADR-003: Documentation L10n Maintenance Pattern](#)
- [ADR-004: Modular Monolith In A Single Monorepo](#)
- [ADR-005: Zero External Dependency Posture](#)
- [ADR-006: `cmd/rgb` As The Unified CLI](#)
- [ADR-007: Bundle Expansion Requires A Consumer Contract First](#)
- [ADR-008: Retire The Go HTML/Print/PDF Publication Pipeline](#)
- [ADR-009: Retain The MkDocs/Python Publication Exception Until Replacement Parity](#)
- [ADR-010: Go-Owned Publication Without Python](#)
- [ADR-011: Split License Model](#)
- [ADR-012: MIT-Only License](#)
- [ADR-013: Skill Distribution Via The Existing Publication Pipeline](#)
- [ADR-015: Skill Runtime Split — LLM Instruction Layer vs. Go Component Layer](#)
- [ADR-016: Reinstate A Narrow External PDF Authoring Exception](#)

# Explosives

Explosives represent **area-effect weapons** used to deal damage or create tactical effects in the RGB System.

Grenades and explosive devices can cause direct damage, control enemy movement, or alter the battlefield.

Unlike conventional weapons, explosives usually **affect multiple targets within an area**.

## Detonation Types

Grenades may detonate with different activation delays.

| Detonation Type | Timing         |           |
|-----------------|----------------|-----------|
| -----           | -----Immediate | same turn |
| Short Delay     | 1 turn         |           |
| Delayed         | up to 2 turns  |           |

The detonation delay may be chosen by the character when throwing the grenade, depending on the device used.

## Types of Grenades

The RGB System supports multiple types of explosives with different tactical effects.

| Type         | Effect                         |               |
|--------------|--------------------------------|---------------|
| -----        | -----Explosive                 | 8 area damage |
| Electrical   | 2 damage + possible disruption |               |
| Sonic        | 1d5 rounds stunned             |               |
| Immobilizing | 1d5 rounds immobilized         |               |

### Explosive Grenade

- causes physical damage in an area
- damage may be reduced by **armor** or **cover**

## Electrical Grenade

- causes low direct damage
- may interfere with electronic equipment or energy systems

## Sonic Grenade

- minimal physical damage
- causes **temporary stun effects**

## Immobilizing Grenade

- temporarily prevents movement
- used for **area control**

## Interaction with the RGB System

Explosives interact with RGB vectors in the following ways:

R → may influence throwing power or accuracy  
G → affects positioning and evasion from the blast area  
B → may absorb part of the damage through shields

The Game Master may adjust these interactions depending on the setting.

## Interaction with the Damage Model

When a grenade deals direct damage, the resolution follows the RGB damage flow:

Grenade Damage  
↓  
Penetration (if applicable)  
↓  
Armor Reduction  
↓  
Shield Absorption  
↓  
Remaining Damage → Character

## Tactical Use

Explosives are particularly useful for:

- area denial
- dispersing groups of enemies
- forcing enemies out of cover

- disrupting defensive positions

Because they affect multiple targets, explosives can **rapidly change the dynamics of a battle**.

## Design Philosophy

Explosives in the RGB System follow three principles:

- **high tactical impact** – quickly alter battlefield conditions
- **simple rules** – minimal values for fast resolution
- **modularity** – new explosive types can easily be added depending on the setting

## See Also

- [Firearms](#)
- [Damage Model](#)
- [Attack and Defense](#)

← [Back to README](#)

# Firearms

Firearms represent the **primary ranged weapons** used in modern and tactical RGB settings.

Unlike melee weapons, firearms **do not add the user's R (Red) vector to damage**.  
Their damage is determined by the weapon itself, representing ballistic force and ammunition power.

Firearms interact with the **RGB Combat System** and follow the standard **Damage Model** used throughout the system.

## Firearm Statistics

Each firearm is defined by four attributes:

- Damage

Capacity

Reload

Type
- base damage inflicted by the weapon

→ number of rounds in the magazine

→ actions required to reload

→ category of weapon

## Firearms Table

| Type              | Damage | Capacity | Reload       |
|-------------------|--------|----------|--------------|
| -----             | -----  | -----    | -----        |
| 12 Compact Pistol | 4      | 15       | minor action |
| Pistol            | 4      | 15       | minor action |
| Revolver          | 5      | 6        | action       |
| SMG               | 5      | 30       | action       |
| Assault Rifle     | 7      | 30       | action       |
| Heavy Rifle / DMR | 8      | 20       | action       |
| Sniper Rifle      | 9      | 10       | action       |
| Machine Gun       | 7      | 100      | 2 actions    |

## Weapon Roles

Different firearms fill different tactical roles.

## Pistols

- lightweight sidearms
- quick to reload
- suitable for close combat

## SMGs

- high rate of fire
- effective at short-to-medium range

## Assault Rifles

- balanced damage and capacity
- effective in most combat situations

## Heavy / Precision Rifles

- higher damage
- lower ammunition capacity
- specialized roles

## Machine Guns

- sustained suppressive fire
- large ammunition capacity
- slower reload time

## Interaction with the RGB System

Firearms interact with RGB vectors indirectly.

R → improves offensive abilities or weapon handling  
G → influences positioning and tactical movement  
B → provides defensive protection through shields

However, firearm damage **comes from the weapon**, not the character's R value.

## Interaction with the Damage Model

When a firearm attack hits a target, the RGB damage flow applies:

```
Impact Source
↓
Penetration (if applicable)
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

This keeps firearm combat consistent with the rest of the RGB system.

## Design Philosophy

Firearms in RGB follow three principles:

- **clarity** — simple weapon statistics
- **tactical variety** — different weapons fulfill different roles
- **system compatibility** — weapons integrate with the RGB damage model

This allows firearms to remain powerful while keeping combat resolution fast and consistent.

## See Also

- [Combat](#)
- [Weapon Ranges](#)
- [Damage Model](#)

← [Back to README](#)

# Melee Weapons

Melee weapons represent **weapons used in close combat** in the RGB System.

Unlike firearms, melee weapons **use the character's R (Red) vector as the base for damage**.

This represents physical strength, impact force, and martial skill.

## Damage Calculation

The damage of a melee weapon is calculated by adding the character's **R value** to the weapon bonus.

Damage = R + weapon bonus

Example:

R = 3  
Long Sword = +3  
  
Total Damage = 6

## Melee Weapons Table

| Weapon      | Damage Bonus |    |
|-------------|--------------|----|
| -----       | -----Knife   | +1 |
| Dagger      | +1           |    |
| Short Sword | +2           |    |
| Long Sword  | +3           |    |
| Axe         | +3           |    |

## Interaction with the RGB System

Melee weapons are directly linked to the **R (Red) vector**.

R → determines base damage  
G → influences mobility and positioning  
B → provides additional defense through shields

Characters with high **R** values tend to be more effective in melee combat.

## Interaction with the Damage Model

When a melee attack hits a target, the RGB damage resolution flow applies.

```
Damage (R + weapon)
↓
Penetration (if applicable)
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

## Tactical Use

Melee weapons are particularly effective in:

- close combat situations
- confined spaces
- environments where firearms cannot be used

They also allow silent attacks and ambush tactics.

## Design Philosophy

Melee weapons in RGB follow three principles:

- **simplicity** — direct damage calculation based on R
- **balance** — damage proportional to the character's strength
- **system compatibility** — direct integration with the RGB damage model

## See Also

- [Combat](#)
- [Damage Model](#)
- [Firearms](#)

← [Back to README](#)

# Penetration

Penetration represents a weapon's ability to **overcome armor protection** in the RGB System.

Weapons with higher ballistic power or physical impact have greater ability to penetrate armor.

## Penetration Calculation

Penetration temporarily reduces the target's armor value during damage calculation.

$$\text{Effective Armor} = \text{Armor} - \text{Penetration}$$

Example:

```
Armor = 4
Weapon Penetration = 2

Effective Armor = 2
```

If the effective armor value reaches **0 or less**, the armor does not reduce damage.

## Weapon Penetration Table

| Weapon        | Penetration |
|---------------|-------------|
| -----Pistol   | 1           |
| SMG           | 2           |
| Carbine       | 2           |
| Assault Rifle | 3           |
| Sniper Rifle  | 4           |
| Machine Gun   | 3           |

## Interaction with the Damage Model

Penetration is applied **before armor damage reduction**.

Full damage resolution flow:

```
Impact Source
↓
Penetration
↓
Effective Armor
↓
Shield Absorption
↓
Remaining Damage → Character
```

## Interaction with the RGB System

Penetration is primarily related to weapon impact, but it indirectly interacts with RGB vectors.

```
R → increases damage for melee weapons
G → allows characters to avoid attacks through mobility
B → provides additional protection through shields
```

Weapons with higher penetration are more effective against heavily armored targets.

## Design Philosophy

The penetration system in RGB follows three principles:

- **simplicity** — direct and easy-to-apply calculation
- **balance** — different weapons have different effectiveness against armor
- **compatibility** — direct integration with the RGB damage model

This allows weapons to be differentiated without adding excessive complexity to combat.

## See Also

- [Firearms](#)
- [Damage Model](#)
- [Armor](#)

← [Back to Readme](#)

# Weapon Ranges

Weapon range defines **the effectiveness of ranged weapons at different combat distances** in the RGB System.

These rules represent the loss of precision and effectiveness as the target moves farther away.

## Distance Modifiers

Attacks with ranged weapons receive modifiers depending on the distance to the target.

| Distance | Modifier    |
|----------|-------------|
| -----    | -----Short0 |
| Medium   | -1 R        |
| Long     | -2 R        |

These modifiers represent the increasing difficulty of hitting distant targets.

## Range Table (meters)

| Weapon        | Short | Medium | Long             |    |    |    |
|---------------|-------|--------|------------------|----|----|----|
| -----         | ----- | -----  | --Compact Pistol | 10 | 20 | 35 |
| Pistol        | 15    | 30     | 50               |    |    |    |
| Revolver      | 20    | 40     | 60               |    |    |    |
| SMG           | 20    | 50     | 80               |    |    |    |
| Assault Rifle | 40    | 120    | 250              |    |    |    |
| Heavy Rifle   | 60    | 200    | 400              |    |    |    |
| Sniper Rifle  | 150   | 500    | 1000             |    |    |    |
| Machine Gun   | 60    | 200    | 400              |    |    |    |

## Extreme Range (Optional)

Attacks beyond long range may still be attempted, but they are significantly more difficult.

|                      |
|----------------------|
| Extreme Range → -4 R |
|----------------------|

The Game Master decides whether the target is still visible and whether the shot is plausible.

## Interaction with the RGB System

Range interacts mainly with the **R (Red) vector**, which represents offensive precision.

R → accuracy and effectiveness of attacks  
G → positioning to obtain better line of sight  
B → defensive protection against attacks

Characters with higher **R** values tend to handle distance penalties more effectively.

## Interaction with the Combat System

Range modifiers are applied during the attack step.

Simplified resolution flow:

```
Attack
↓
Apply distance modifier
↓
Resolve defense
↓
Apply damage model
```

This keeps the system consistent with:

- [Attack and Defense](#)
- [Damage Model](#)

## Design Philosophy

The RGB range system follows three principles:

- **simplicity** — few modifiers that are easy to remember
- **consistency** — the same model for all ranged weapons
- **tactical balance** — different weapons have different effective ranges

This ensures that positioning has real impact during combat.

## See Also

- [Firearms](#)
- [Penetration](#)
- [Damage Model](#)

← [Back to Firearms](#)